

СИСТЕМА ИДЕНТИФИКАЦИОННЫХ ТОКЕНОВ · ВЕРСИЯ 3



POLARIS

Fixus inter mutabilia

Карта системы

Справочная реализация системы идентификационных токенов,
несвязываемой со стороны эмитента и учитывающей принуждение, подписанной
ML-DSA-65 по проверяемому пути миграции алгоритмов; читается от центра
наружу, в версии 1.0.0-rc.7

Егор Хаклин

УНИВЕРСИТЕТ СЕТОН-ХИЛЛ · СЕНТЯБРЬ 2026

О настоящей версии

Версия 1 настоящего отчёта (`polaris_project_report.pdf`, сохранена без изменений рядом с этим файлом) описывала Polaris как проект базы данных: двенадцать таблиц вокруг одного удостоверения, модель «сущность–связь», автомат жизненного цикла и довод о том, что идентификационные данные обязаны быть постквантовыми с первого дня. Версия 2 (`polaris_project_report_v2.pdf`, также сохранена) описывала систему в том виде, в каком она существовала на теге `v9.345`: тридцать шесть таблиц, протокол федерации, слой прозрачности и ткань межведомственного обмена; она была пересмотрена на `v9.355`, когда работа на стороне держателя, отмеченная в ней как отсутствующая, была выполнена. Версия 3 описывает систему на версии `1.0.0-rc.7` (коммит `4e74376`, 20 сентября 2026 года), измеренную заново целиком, а не залатанную, и написана она для читателя, который никогда не открывал репозиторий. Впервые она была написана на `1.0.0-rc.1` четырьмя днями ранее и перемерена на месте; четырнадцать из двадцати шести величин в приложении ?? не сдвинулись, а те, что сдвинулись, там отмечены.

Что изменилось, в одном абзаце. Между версией 2 и настоящей репозиторий перестал считать собственный прогресс и начал считать свидетелей. Четыре самостоятельных пакета, которые устанавливает доверяющая сторона, находятся в публичных реестрах. Кошелёк, написанный посторонними людьми и запущенный без изменений, предъявил удостоверение верификатору и был принят, после того как один раз получил отказ из-за дефекта, мимо которого прошёл каждый внутренний инструмент. Размещённый пакет проверок соответствия OpenID Foundation прогнал профиль верификатора через открытый интернет. Сам слой проверки подвергся атаке изнутри: инвариантные проверки, ограничения, триггеры, уникальные индексы, хранимые процедуры, свидетельства с нулевым разглашением, контракт соответствия и оба эталонных верификатора были поочерёдно мутированы и обязаны были покраснеть, и многие поначалу этого не сделали. Физический слой стал предметом, который способен изготовить кто-то другой. И два слова на парадной двери были ослаблены: *постквантовый до алгоритмической гибкости при проверяемом пути миграции, а устойчивый к принуждению до осведомлённого о принуждении*, потому что имеющиеся свидетельства не подтверждали более сильные формулировки. Приложение ?? перечисляет каждое утверждение версии 2, которое настоящая версия отзывает или сужает.

Три даты, названные прямо. Основной текст версии 2 был измерен на `v9.345`; её разделы о стороне держателя на `v9.355`. Всё в настоящей версии, каждая величина, каждая метка и каждый рисунок, измерено на `1.0.0-rc.7` методами из приложения ?? Там, где число старше этого, потому что репозиторий не перезапускал породивший его инструмент, документ говорит об этом рядом с числом. Ничто из работавшего на `v9.355` работать не перестало.

Документ сохраняет то, что версия 1 угадала верно: геометрию схемы с токеном в центре, используемую здесь как центр каждой карты, и автомат жизненного цикла; а также то, что верно угадала версия 2: концентрическое прочтение системы, по одному кольцу на раздел, где каждое кольцо вводится отказом кольца внутри него.

Пять меток

Каждая способность несёт одну из пяти меток, чтобы работающий код никогда не путали с намерением, а то, что проверил автор, никогда не путали с тем, что проверил кто-то ещё.

РЕАЛИЗОВАНО Путь, который выполняется на `1.0.0-rc.7`, задействован тестом или учением в непрерывной интеграции и закреплён инвариантной проверкой, роняющей сборку, если он перестаёт быть истинным.

ЭКСПЕРИМЕНТАЛЬНО Путь, который выполняется, но ограничен способом, который называет сам репозиторий: не покрыт CI, условен по масштабу или зависит от серверной части, которую CI имитирует.

ТРЕБУЕТСЯ ВНЕШНЯЯ ПРОВЕРКА Свойство, которое репозиторий реализует и проверяет внутренними средствами, но которое никакая внешняя сторона не проверяла аудитом, не атаковала, не разворачивала и не реализовывала независимо заново.

ПРОВЕРЕНО ИЗВНЕ Путь, который названная сторона за пределами репозитория действительно задействовала, в конкретную дату, с результатом, на который можно указать: кошелёк, служба проверки соответствия, опубликованный набор тестов. Метка намеренно редка, она даётся только там, где в собственной сводной таблице репозитория заполнена строка, и она никогда не означает аудит, сертификацию или пилотное внедрение. В настоящем документе её несут три вещи.

ПРЕДЛОЖЕНО Предложение: строка дорожной карты, проектная записка или замысел, существующий только в настоящем документе. Это не код.

Строка, несущая две метки, работает сегодня и всё же требует внешней проверки для более сильного утверждения либо была задействована извне на одном пути и не была на остальных.

Числа измерены на дереве на 1.0.0-rc.7 методами из приложения ??, а не переписаны из более ранних документов. Там, где собственные документы репозитория расходятся с его кодом, побеждает код, а расхождение фиксируется в авторских заметках, сопровождающих настоящую версию.

Аналогия и её предел

Документ объясняет Polaris через одну сквозную аналогию: город, обнесённый стеной. Запись удостоверения есть реестр городской ратуши; ограничения есть стена; конечные точки есть ворота; журналы прозрачности есть сторожевые башни; независимые свидетели стоят за стеной и наблюдают за башнями; другие удостоверяющие органы есть другие города, соединённые дорогами с односторонним движением; над всем городом стоят законы, связывающие и сам город; и, впервые в настоящей версии, к воротам пришли путники издалека, и найденное ими есть единственный отчёт, который город написал не о себе сам. Аналогия описывает расположение и линии обзора, а не политическую централизацию: Polaris федеративен в пределах каждого удостоверяющего органа, без центральной базы данных, корня или службы проверки, и всякое возвращение аналогии возвращается к реальному механизму под его настоящим именем.

Один тезис и одно правило

Система есть попытка построить идентификационную инфраструктуру, в которой важным утверждениям не приходится доверять лишь потому, что центральное приложение называет их истинными. Полномочия, состояние, история, границы приватности и собственное поведение системы последовательно выставляются на независимую проверку и стесняются слоями, которые не разделяют допущений друг друга. Каждый слой существует потому, что слой внутри него может быть совершенно верен и всё же не доказать того, что обществу необходимо знать. Эта последовательность есть хребет документа.

Правило, принятое 13 сентября 2026 года и применяемое ко всему с тех пор, состоит в том, что единицей прогресса является новая пережитая внешняя зависимость: названный кошелёк, названный профиль соответствия, названная доверяющая сторона, дефект, заведённый кем-то, кто не является автором. Число инвариантов, версий и строк прогрессом не является, и сводная таблица репозитория отказывается его записывать. Настоящий документ следует

этому правилу. Там, где он сообщает внутренние свидетельства, он говорит об этом и держит эти свидетельства вне предложений, сообщающих свидетельства другого рода.

Содержание

О настоящей версии	i
1 Задача	1
2 Как читать эту карту	3
3 Ядро: одно удостоверение, одна подпись, один человек	5
4 Стены: схема как граница безопасности	10
5 Удостоверение истины: подписи, хранение ключей, два свидетеля и одна миграция	16
6 Проверка без эмитента: пакетные верификаторы, контракт соответствия и первые посторонние	21
7 Проверка не есть полномочия: состояние, отзыв и размен автономности	26

1 Задача

Человек в Соединённых Штатах носит с собой от шести до восьми удостоверяющих документов, которые друг с другом не разговаривают: водительское удостоверение, паспорт, карточку социального страхования, Real ID, регистрацию избирателя, страховую карту. Каждый из них есть отдельный предмет, подписанный отдельным органом по отдельному стандарту, без общего пути отзыва и без общего следа для аудита. Версия 1 настоящего отчёта бралась смоделировать их объединение в одну действующую запись удостоверения на человека, проверяемую через события с областью действия, заданной контекстом. Это лёгкая половина задачи, и она по-прежнему задаёт её очертания.

Тяжёлая половина состоит в том, что происходит, когда система работает. Идентификационная система, добившаяся успеха, становится дверью ко всему, а дверь есть место, где сосредоточивается власть. Отсюда следуют три режима отказа, и каждый слой Polaris существует, чтобы ответить на один из них.

Система лжёт о прошлом. Оператор или инсайдер с доступом к базе данных отзываёт токен, а позже стирает запись; выдаёт удостоверение по процедурам, о которых обществу не говорили; или показывает одну историю аудитору и другую суду. Национальной идентификационной системе нельзя позволять задним числом переписывать то, что она сделала.

Система наблюдает. Каждая проверка есть факт о том, где человек был и чем занимался. Журнал таких фактов есть хребет слежки, независимо от того, задумывал ли его кто-нибудь, и он остаётся им после того, как намерения оператора переменятся.

Система принуждает. Держателя можно заставить: пограничник, работодатель или вор могут вынудить предъявить удостоверение, а дверь, которой требует каждая служба, способна вынудить население зарегистрироваться. Идентификационная система есть также и самый действенный инструмент исключения из всех когда-либо построенных, если список тех, кого в ней нет, можно получить по требованию.

Общепринятый ответ на все три есть политика: правила о сроках хранения, доступе, аудите и согласии, применяемые теми людьми, которые систему и эксплуатируют. Конституция Polaris (MISSION.md) держится позиции, что политика ничего не стоит как единственная опора обещания, потому что люди, эксплуатирующие систему, суть ровно те, кого обещание и призвано связать. Обещание должно быть закреплено там, где его нельзя обойти разговором: в схеме, в триггерах, в уникальных индексах, в ограничениях целостности, в подписях, которые любой может проверить, ни о чём систему не спрашивая, и в журналах, за которыми наблюдают посторонние. Приложение не есть граница безопасности. Ею является база данных, а слои вокруг базы данных существуют затем, чтобы даже её собственные утверждения могли проверить посторонние.

1.1 Почему постквантовость с первого дня и что этой фразе позволено означать

Версия 1 доказывала через неравенство Моски, что горизонт секретности идентификационных данных измеряется поколениями, что миграция национальной системы занимает десятилетие и что система удостоверений, развёрнутая на классической криптографии в 2026 году, есть поэтому отложенная компрометация, а не работающая система. Этот довод не изменился и кратко изложен в приложении ???. На 1.0.0-rc.7 алгоритмом подписи по умолчанию является ML-DSA-65 (FIPS 204), рядом с ним принимается ML-DSA-87, система доказательств с нулевым разглашением построена на хеш-функциях и не требует доверенной настройки,

а публичная кромка TLS и один из двух внутренних переходов согласуют гибридный постквантовый обмен ключами.

То, что репозиторий об этом утверждает, сузилось со времён версии 2, и само это сужение есть один из предметов настоящей версии. Слово *постквантовый* точно в отношении алгоритмов. Оно не является утверждением, которое репозиторий вправе делать о безопасности, потому что стойкость ML-DSA опирается на решёточные допущения, которые могут пасть от математики, а не от железа, и потому что путь разработки по умолчанию подписывает помеченной заглушкой, а вовсе не решёточной подписью. Защитимое утверждение звучит так: *алгоритмическая гибкость при проверяемом пути миграции*. Алгоритм есть поле в схеме и никогда не константа, население можно переподписать по новому алгоритму путём, который выполняется при каждой отправке, и во что обошёл бы взлом, записано. Раздел 5 приводит довод и цену; инвариантная проверка отказывает любой внешней поверхности, говорящей больше.

1.2 Последовательность, которой следует настоящий документ

Каждое из следующих предложений называет слой, и каждое истинно в отношении слоя, стоящего перед ним.

- Подлинность удостоверения не есть его нынешние полномочия. Подпись остаётся настоящей навсегда; полномочия под ней могут быть отозваны сегодня после обеда.
- Нынешние полномочия не есть приватность. Верификатор может узнать, что удостоверение действительно, и всё же узнать куда больше, чем ему требовалось.
- Приватность не есть свобода от принуждения. Система может не знать о человеке почти ничего и всё же оставаться дверью, которой требует каждая служба.
- Федерация не есть институциональная независимость. Два экземпляра одного и того же кода, запущенные одним и тем же автором, доказывают, что протокол работает, а не что его эксплуатируют независимые учреждения.
- Журнал прозрачности не есть защита от расщеплённого обзора. Одному наблюдателю могут показать частное ответвление истории, и он об этом не узнает.
- Зелёный тест не есть доказательство того, что тест задал верный вопрос.
- Система, проверившая саму себя, не есть система, которую проверил кто-то ещё. Каждый инструмент в репозитории прошёл мимо дефекта, который первая же сторонняя реализация отвергла с первого взгляда.

Последние два предложения дались проекту дороже всего. Раздел ?? о шестом, а раздел 6 несёт седьмое.

2 Как читать эту карту

Polaris не есть одна огромная таблица или один огромный сервер. Это очень небольшое ядро удостоверения с намеренно ограниченными структурами, расставленными вокруг него, каждая из которых стесняет, защищает, наблюдает, проверяет, записывает или ограничивает что-то в структуре, лежащей внутри неё. Документ читается от ядра наружу, кольцо за кольцом (рис. 1), и читатель должен иметь возможность менять увеличение: от человека к удостоверению, к криптографии, которая его удостоверяет, к правилам схемы, которые его ограничивают, к органу, который его выдал, к сторонам, которые на него полагаются, к федерации органов, к свидетелям, которые за ними наблюдают, к межведомственному обмену, к операциям, которые поддерживают в нём жизнь, к обществу, внутри которого он находится, к среде программных агентов, которую он теперь обслуживает, и наконец наружу: к программам, написанным посторонними, чьё принятие или отказ есть единственное свидетельство, которое репозиторий произвёл не о себе сам.

2.1 Вопросы, задаваемые каждому слою

О каждой крупной составной части документ отвечает на одни и те же вопросы, в карточке того вида, который используется повсюду. Что это такое? Почему оно существует? Какую задачу оно решает? Чему оно доверяет? Чему оно доверять отказывается? Что оно видит? Чего ему намеренно не дают видеть? Что происходит, если оно отказывает? Какая часть Polaris его проверяет? Нужно ли оно сейчас или это подготовка к будущей среде? Как оно соединено со слоем внутри него и со слоем снаружи? Читатель, способный ответить на эти вопросы про каждое кольцо, владеет той мысленной моделью, ради построения которой настоящий документ и существует.

2.2 Город

IN THE TOWN Запись удостоверения есть реестр, хранимый в городской ратуше: по одной записи на человека, утверждение, вокруг которого устроено всё остальное. Стена есть схема: триггеры, ограничения целостности и уникальные индексы, решающие, какие состояния вообще вправе существовать, и связывающие одинаково каждого посетителя, каждого чиновника и каждое восстановление из резервной копии. Ворота есть конечные точки, через которые утверждение входит и выходит: выдача, проверка, вход, обмен. Сторожевые башни есть журналы прозрачности, а свидетели стоят за стеной, совместно подписывая то, что башни публикуют, и поднимая крик, когда две башни расходятся. Другие города есть другие удостоверяющие органы, у каждого своя ратуша и свой ключ, и соединены они дорогами, которые идут в одну сторону и ради одной цели. Надо всем стоят законы, которых город не в силах отменить голосованием своих чиновников. И у главных ворот, начиная с настоящей версии, путник из города, которого здесь никто не строил, предъявил бумаги и был пропущен, после того как сперва получил поворот от ворот из-за изъяна в собственной печати города, которого не заметил ни один из его писарей.

Карта города, с техническим именем рядом с каждым городским, есть рис. 5 в разделе 4. Аналогия будет возвращаться в затенённой врезке вроде помещённой выше, и за каждым возвращением следует механизм, который она обозначает. Путник есть кошелёк `walt.id`, ворота есть верификатор `OpenID4VP`, а изъян в печати был сертификатом для подписи запросов без назначения ключа `digitalSignature`; раздел 6 называет все три по имени.

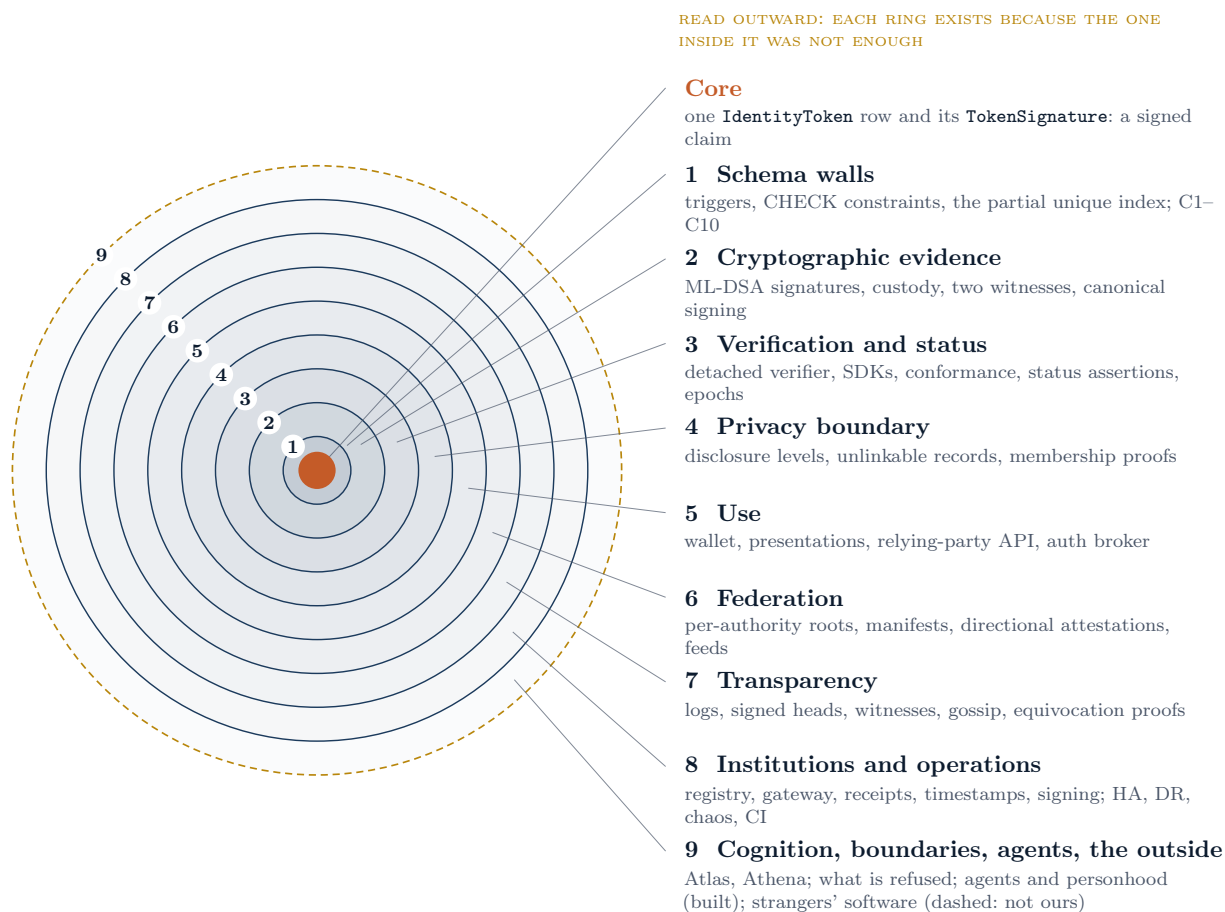


Рис. 1: Концентрическая карта Polaris. Ядро есть одна запись удостоверения и её подпись. Каждое кольцо называет действительные составные части на нём, а порядок чтения документа следует кольцам наружу. Внешнее кольцо показано пунктиром, потому что по большей части оно не код: оно есть то, что стороны за пределами репозитория сделали с кольцами внутри него, а на **1.0.0-rc.7** это один кошелек, один размещённый прогон проверки соответствия и один опубликованный набор тестов.

2.3 Доверие и отказ

Самая полезная привычка при чтении Polaris состоит в том, чтобы спрашивать о каждом слое, что именно он отказывается принимать на веру от слоя под ним. Схема отказывается доверять приложению. Отделяемый верификатор отказывается доверять серверу эмитента. Второй свидетель отказывается доверять первой реализации. Доверяющая сторона отказывается доверять транзитивной цепочке удостоверяющих органов. Свидетель отказывается доверять собственной вершине журнала. Слой инвариантных проверок отказывается доверять проверке, которая не способна обнаружить собственное нарушение. Мутационные учения отказываются доверять тесту, которому ни разу не показали, как он падает. А сводная таблица отказывается засчитывать что бы то ни было, что автор прогнал против собственного кода. Приложение ?? сводит эти отказы в лестницу; разделы 3 по ?? объясняют каждый из них.

3 Ядро: одно удостоверение, одна подпись, один человек

3.1 Что такое удостоверение

В центре Polaris находится одна строка таблицы `IdentityToken` и по меньшей мере одна строка `TokenSignature`. Строка токена записывает уникальное `token_value`, физический серийный номер, человека, которому он принадлежит, ведомство, которое его выдало, алгоритм, под которым он был выдан, его предшественника, если он заменил собой более ранний токен, и его состояние. Строка подписи записывает подпись ML-DSA выдавшего ведомства над дайджестом SHA3-256 значения токена вместе с открытым ключом, которым она была произведена, так что проверка самодостаточна и переживает вывод подписывающего ключа из обращения. Токен без строки подписи существовать не может: триггер базы данных отвергает любую запись, которая оставила бы токен без неустаревшей подписи. Удостоверение есть поэтому не документ, а подписанное утверждение: *этот орган выдал этот токен этому человеку*. Начиная с v9.349 утверждение может также связывать ключ, которым распоряжается держатель, записанный в `HolderKeyEvent` под той же дисциплиной добавления без изменения, что и ключ удостоверяющего органа, так что владение файлом перестаёт быть владением удостоверением (раздел ??). **РЕАЛИЗОВАНО**

Держатель держит у себя пакет подлинности: значение токена, подпись и открытый ключ эмитента, выгружаемые приложением и хранимые в виде файла эталонным кошельком (`scripts/polaris-wallet.py`). Любой может проверить этот пакет стандартной библиотекой ML-DSA и без единой строки кода Polaris (раздел 6).

3.2 Физический токен как предмет, который способен изготовить кто-то другой

Версия 2 записала аппаратную карту как смоделированную и не произведённую. Это по-прежнему верно в отношении кремния и неверно в отношении всего, что его окружает. Начиная с v9.366 карта есть кодировка с эталонным кодеком и опубликованными векторами (`polaris_card/card_profile.py`), программный токен, говорящий на ISO 7816-4 настоящими словами состояния, с двумя гнездами для PIN, счётчиком попыток и PUK (`emulator.py`), шаг персонализации, на котором карта сама вырабатывает свои ключи, так что орган никогда ими не владеет, и эталонное устройство проверки, которое читает карту по NFC либо предъявление по QR-коду и сообщает три вывода по отдельности: владение, подлинность и полномочия, потому что отозванная карта всё равно подпишет вызов, а устройство, говорящее лишь *принято*, ничему своего оператора не учит. Профиль прямо называет своё единственное формообразующее ограничение: сертифицированные защищённые элементы, подписывающие по FIPS 204 аппаратно, сегодня не есть нечто, что орган может закупить массово, поэтому карта несёт две подписи эмитента над одним телом, классическую, которую нынешний кремний произвести способен, и ML-DSA, а считыватель решает, какой из них доверяет. Каждая из этих вещей выполняется в учении при каждой отправке. **РЕАЛИЗОВАНО** для профиля, эмулятора, векторов и устройства; **ПРЕДЛОЖЕНО** для произведённой карты, чего никакое учение установить не может.

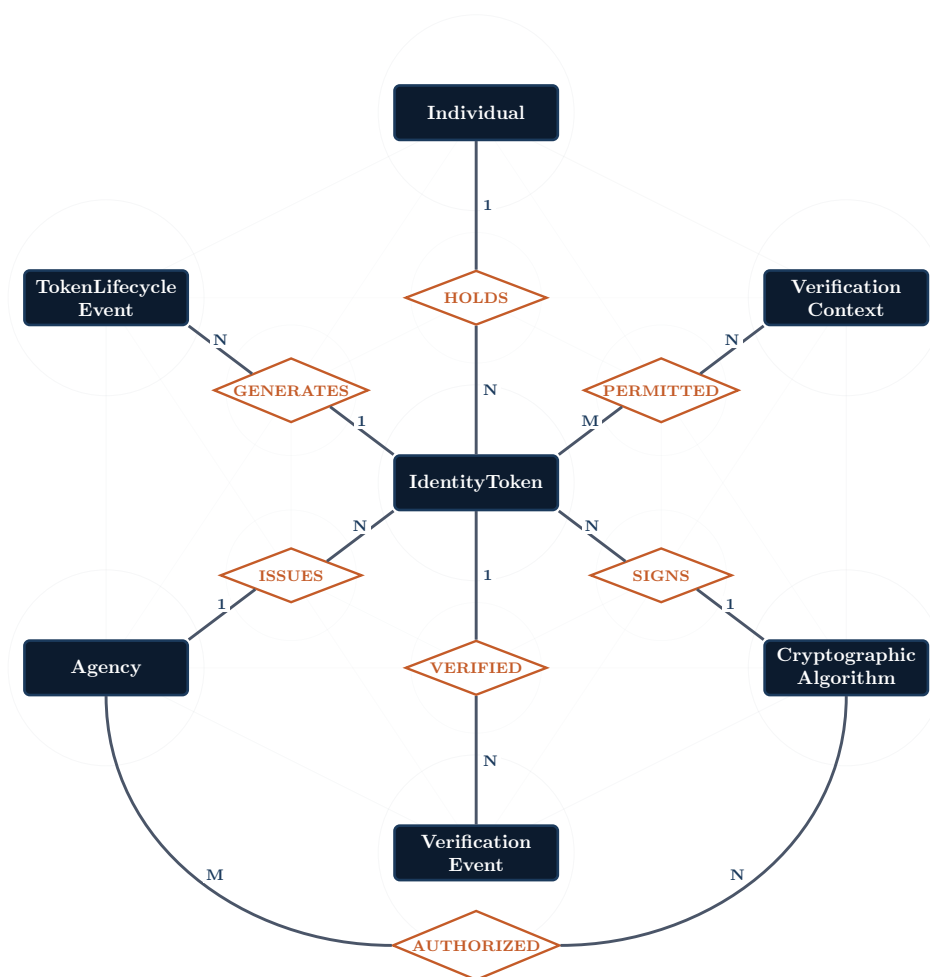


Рис. 2: Геометрия сущностей ядра, не изменившаяся с версии 1. Семь сущностей вокруг **IdentityToken**: человек, который его держит, ведомство, которое его выдаёт, алгоритм, который его подписывает, контексты, в которых он может проверяться, и два журнала событий, работающих только на добавление, которые он порождает. Связи «многие ко многим» разрешаются в соединительные таблицы **AgencyAlgorithmAuth** и **TokenPermission**. Эта геометрия есть постоянная величина документа: раздел 4 рисует остальные тридцать восемь таблиц как кольца вокруг неё.

3.3 Чем удостоверение не является

Десятое ограничение конституции, C10, гласит, что удостоверение личности никогда не несёт платёжных полномочий: в схеме нет ни одного денежного поля, и инвариантная проверка роняет сборку, если такое появится. Причина изложена как режим отказа, а не как предпочтение. Удостоверение, которое есть ещё и деньги, наследует программируемость: всякое ограничение, какое кому-либо хочется наложить на траты, нарастает на идентификационный токен, покуда административная ошибка в бумагах не станет экзистенциальной ошибкой в банковском балансе и покуда системе нельзя будет сказать, что этому человеку нельзя покупать топливо. Тот же отказ покрывает репутацию, поведение и профили. Polaris отвечает на один вопрос, *действителен ли этот токен для этого контекста прямо сейчас*, и уклоняется от ответа на вопрос *следует ли позволить этому человеку сделать X*. Вторым вопросом принадлежит доверяющей стороне, за границей. **РЕАЛИЗОВАНО**

3.4 Один действующий токен на человека

Человек вправе держать несколько подготовленных токенов, но лишь один может быть **ACTIVE**. Два действующих токена у одного человека позволили бы одному токenu разрешить операцию, а другому от неё отпереться. Это правило C3, и оно закреплено частичным уникальным индексом `uq_one_active_per_person` по идентификатору человека там, где состояние есть действующее. Два оператора, активирующие два резервных токена для одного держателя в один и тот же миг, обнаруживают, что удаётся ровно одному из них, и тест, который это доказывает, работает настоящими потоками против живой базы данных (C9). Правило также смоделировано в TLA+, и модель проверяется при каждой отправке, при всяком чередовании одновременных выдачи и отзыва, вместе с парной конфигурацией, на которой она обязана упасть. Верно про него и ещё одно, а раздел ?? рассказывает это целиком: когда индекс убрали, чтобы посмотреть, кто это заметит, выяснилось, что набор тестов, существующий ради доказательства C3, сам его и нарушал, фиксируя второй действующий токен перед тем, как упасть. **РЕАЛИЗОВАНО**

3.5 Жизненный цикл

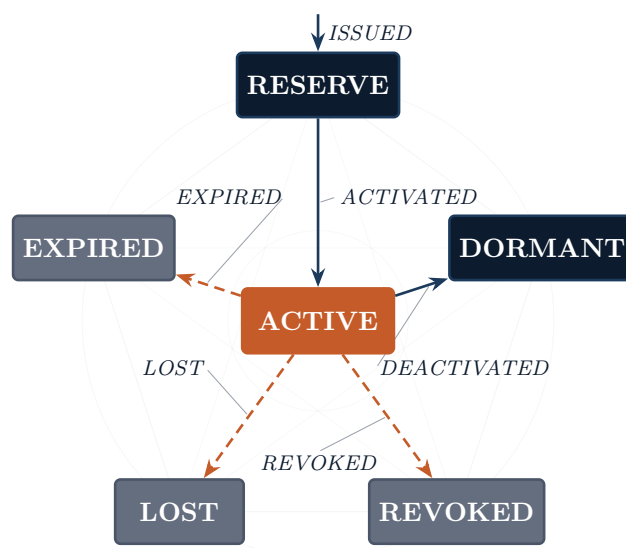


Рис. 3: Автомат жизненного цикла токена, не изменившийся с версии 1. **ACTIVE** есть единственное рабочее состояние. Токен входит через **RESERVE**, активируется и уходит через одно из трёх конечных состояний либо в **DORMANT**, когда дело принимает преемник. Законные переходы закреплены триггером `trg_token_state_machine`, и каждый переход автоматически записывает событие жизненного цикла.

Версия 1 советовала закрепить переходы в триггере и оставила это на будущее. Триггер существует, а второй триггер записывает строку `TokenLifecycleEvent` на каждую смену состояния, включая смену, сделанную вручную из консоли базы данных. Преемство идёт по ссылке, а не перезаписью: замещающий токен указывает на своего предшественника, а старый токен остаётся в базе данных в своём конечном состоянии. Утраченные токены остаются утраченными, и их данные не стираются. **РЕАЛИЗОВАНО**

3.6 Вход, выход и возвращение

Три механизма окружают ядро, чтобы схему нельзя было обратить против держателя на краях жизненного цикла, и у каждого со времён версии 2 выросла запись о том, на чём он держался.

Вход: многоуровневая регистрация и то, на чём держится запись. У каждого человека есть состояние регистрации из пяти значений, и ЕХЕМРТ есть полноправное из них: признанный способ участвовать в гражданской жизни без токена, будь то по биометрической несовместимости, религиозному освобождению или по иному пути. Запись об освобождении есть одна строка; перечислить незарегистрированных возможно, но ни одна функция в этом не помогает, а всякое такое перечисление заносится в аудит. До v9.371 система могла выдать удостоверение и не имела способа сказать, как именно человек был подтверждён как тот, кем себя называет. `EnrollmentProofing` теперь записывает одно событие подтверждения личности, а `EnrollmentEvidence` то, на чём оно держалось, обе только с добавлением записей, и уровень достоверности *выводится* из свидетельств, а не вбивается оператором; поля для самого документа намеренно нет, а названный набор полей отвергается наотрез, так что запись не способна воссоздать паспорт. Чего это не устанавливает, изложено в пакете для рецензента как ограничение: подлинность свидетельств принимается со слов оператора. **РЕАЛИЗОВАНО**

Всякое сочетание свидетельств, какое называет любой стандарт, начинается с документов, и человек, у которого их нет, выпускник детского дома, беженец, взрослый, никогда не имевший паспорта, не проходит все эти сочетания по умолчанию. Доверенное поручительство (`RefereeVouching`) есть ответ на это, и оно же есть простейший способ вычеканить уровень достоверности из ничего, поэтому репозиторий обращается с механизмом против исключения и каналом подлога как с одной и той же таблицей: поручительство ограничено, совместно подписывается сверх порога и невидимо на самом удостоверении, потому что человек, которому оно понадобилось, не должен носить за это метку у каждой стойки. Регистрационный код, последняя часть, не требующая никакого железа, устанавливает лишь то, что некто, имеющий доступ к каналу, вернул секрет, и уровень, который он способен внести, ограничен соответственно. **РЕАЛИЗОВАНО**

Выход: усмотрение эмитента как решение, переживающее собственное изменение. Худший отказ идентификационной системы есть не поддельный токен, а орган, отзывающий токены у целого населения в промышленном масштабе. Триггер ограничивает скорость, с которой одно ведомство способно отзывать собственные токены (по умолчанию пять процентов действующей базы за тридцать дней), а сверх того требуется соподписант из другого ведомства, заносимый в след аудита. Ограничение превращает неожиданность в тенденцию, которую отчётность видит; оно не останавливает медленного злоупотребления под потолком, о чём и говорит. Со времён версии 2 настройки вокруг этой власти стали записями: сам потолок (`IssuerDiscretionPolicy`) можно изменить только с изложенным обоснованием, и он оставляет строку о том, кто его изменил и с какого значения; квота на то, какую долю населения ведомство вправе затронуть, есть решение с причиной, а не величина, которую можно перезаписать; строка в корне иерархии, `Agency`, несёт триггер, записывающий всякое изменение уровня органа и отмечающий, было ли оно расширением; а планка доверяющей стороны, которую до v9.425 можно было понизить, нигде этого не записав, теперь фиксируется с той же дисциплиной, с пометкой об ослаблении и с обязательными причинами. **РЕАЛИЗОВАНО**

Возвращение: обряд восстановления. Пожар забирает кошелёк вместе с резервом. Восстановление обязательно и вместе с тем есть путь выдать себя за другого, поэтому его делают дорогим, не делая невозможным: двухфазный обряд с сорокавосемичасо-

вым периодом выдержки, три независимых внеполосных канала, утверждающий, который не может быть заявителем, и завершение только администратором. Поля решения движутся в одну сторону под триггером начиная с v9.347, а правило четырёх глаз, период выдержки и три канала суть отказы хранимой процедуры, про которые мутационное учение теперь доказывает, что тесты заметили бы их утрату (раздел 4). Держатель остаётся без личности на время выдержки; замещающее удостоверение, которого промышленная система на это окно захотела бы, не построено. **РЕАЛИЗОВАНО** для обряда, **ПРЕДЛОЖЕНО** для временного удостоверения.

3.7 Принуждение и то, что этому слову позволено означать

Призвание, стоящее над десятью ограничениями, есть противодействие принуждению: никого нельзя заставить отречься от своей личности, передать её или сдать против собственной воли. Механизм, дающий этому предложению опору, стар и пришёл из банковского дела: второй секрет, выглядящий для всякого наблюдающего ровно как успех, в то время как тревога уходит по каналу, которого наблюдающий не видит. Держатель вправе завести код принуждения. Когда он предъявлен, проверка идёт одинаково на каждой видимой оператору поверхности, та же страница, то же всплывающее сообщение, та же строка события, а строка `DuressEvent` добавляется в таблицу, к которой экраны оператора никогда не присоединяются и из которой пейджер срабатывает с наивысшей важностью. Сравнение выполняется за постоянное время, и работа оплачивается на обоих путях. На карте тот же механизм есть второй PIN той же длины, что и первый, вводимый тем же способом и отпирающий второе гнездо ключа, открытого ключа которого объект карты не несёт, а учение измеряет отклик и во времени, и в байтах, потому что принуждающий у считывателя наблюдает, сколько карта провозилась. **РЕАЛИЗОВАНО** как проверенное свойство смоделированного потока.

Версия 2 называла систему устойчивой к принуждению. Версия 3 не называет, потому что собственная оценка репозиторием этого механизма против трёх принуждающих, названных операционным договором, заключила, что слово не подтверждается, и словарь был ослаблен до *осведомлённого о принуждении* на каждой внешней поверхности, с инвариантной проверкой, отвергающей возвращение сильного слова. Против беспечного принуждающего, который не знает о существовании кодов принуждения, механизм работает: экран неотличим, а тревога беззвучна. Против осведомлённого принуждающего, стоящего рядом с держателем, он не работает: код набирается на терминале верификатора, нажатия клавиш не происходят за постоянное время, а поскольку заведение кода добровольно, *у меня нет кода принуждения* есть утверждение, на которое принуждающий может надавить и которого держатель доказать не может. Против последующего законного или ведомственного доступа он хуже, чем ничего: `DuressEvent` работает только на добавление, без пути к очистке, что есть C1, работающее как задумано, и потому там, где принуждающая сторона есть само учреждение или способна позже его принудить, механизм изготавливает неизгладимое свидетельство того, что держатель сопротивлялся. Оценка не предлагает изменения механизма, и настоящий документ не предлагает тоже; он фиксирует границу. Утверждение о побочных каналах, что ничто наблюдаемое не отличает два пути, измерено репозиторием и никем более. **ТРЕБУЕТСЯ ВНЕШНЯЯ ПРОВЕРКА**

Карточка слоя: ядро удостоверения

Что это	Одна строка <code>IdentityToken</code> и её строки <code>TokenSignature</code> , необязательно связанные с ключом держателя: подписанное утверждение, что некий орган выдал этот токен этому человеку, в жизненном цикле с одним действующим состоянием, с записанным рядом подтверждением личности, на котором оно держалось.
Зачем оно существует	Всякая иная структура в Polaris устроивается вокруг этого утверждения. Оно держится настолько малым, насколько возможно, чтобы остальное можно было ограничить.
Чему доверяет	Подписывающему ключу выдавшего ведомства, хранимому за интерфейсом хранения (раздел 5); ограничениям схемы; слову оператора о том, что предъявленные свидетельства были подлинными.
Чему отказывает	Коду приложения как источнику истины: автомат состояний, уникальность и аудит решаются базой данных. Уровню достоверности, вбитому, а не выведенному. Деньгам, репутации и профилям: C10.
Что видит	Законное имя держателя, дату рождения и страну; выдавшее ведомство; алгоритм; жизненный цикл; род свидетельств, на которых держалась регистрация.
Чего не видит	Биометрический шаблон (никогда не хранится), документ, стоящий за регистрацией (поля для него не существуют), граф проверок (раздел ??), историю трат (никогда не моделировалась).
Если откажет	Поддельное ядро есть поддельная подпись, которую отвергают проверка двумя свидетелями и отделяемый верификатор; сдвоенное ядро отвергается уникальным индексом; переписанное ядро отвергается триггерами добавления; принуждающий, который сам есть учреждение, не останавливается, и запись об этом говорит.
Кем проверяется	<code>check_c1c10_objects_resolve</code> , тесты свойств C3 и модель C3, набор тестов на одновременность, <code>check_card_profile</code> , <code>check_card_emulator</code> , <code>check_verifier_device</code> , <code>check_duress_on_card</code> , <code>check_duress_claims_are_aware</code> , случаи подлога и подмены в наборе атак.
Сейчас или потом	Сейчас. Это та часть Polaris, которая существует с версии 1; карта есть предмет уже сейчас, а кремний потом.
Внутри и снаружи	Внутри: человек, существующий за пределами программ. Снаружи: стена ограничений (раздел 4).

IN THE TOWN Городская ратуша держит один реестр. Каждая запись говорит, кто этот человек, какой чиновник её внёс, под какой печатью и какие бумаги чиновнику предъявили, но никогда не хранит самих бумаг. Реестр нельзя править, к нему можно только дописывать; вторая запись о том же человеке отвергается прямо у стойки; а печать такова, что всякий читатель вправе сверить её с опубликованным знаком чиновника, ни о чём ратушу не спрашивая.

4 Стены: схема как граница безопасности

4.1 Почему база данных не есть водопровод

В большинстве систем база данных хранит то, что решило приложение. В Polaris решает база данных, а приложение есть один клиент из нескольких: операторское веб-приложение, инструмент командной строки, массовый загрузчик, консоль базы данных, восстановление из резервной копии. Правило, живущее в коде приложения, связывает лишь тех клиентов, которые этот код выполняют. Правило, закреплённое триггером, ограничением целостности или уникальным индексом, связывает каждого клиента, переживает всякое восстановление и не может быть обойдено следующим вызывающим, потому что у роли приложения нет привилегии определения данных, которой его можно было бы снять. Управляющее предложение всей архитектуры звучит так: *по построению*. Систему невозможно эксплуатировать способом, нарушающим ограничение, потому что ограничение закрепляется до того, как выполнится хоть строка кода приложения. **РЕАЛИЗОВАНО**

Проектный принцип за этим старше Polaris: сделать недопустимые состояния непредставимыми, а не просить разработчиков их не создавать. Событие проверки, записывающее идентификатор токена на уровне раскрытия с нулевым разглашением, есть не ошибка приложения, которую надлежит отловить на ревью. Это нарушение приватности, которое обязано быть нехранимым, и потому оно есть ограничение целостности на строке.

4.2 Десять ограничений и где живёт каждое

Конституция называет десять жёстких ограничений в двух ярусах. Пять суть конституционные, гарантии прав, которые нельзя ослабить без поправки, занесённой в журнал изменений; пять суть инженерные инварианты, дисциплины, удерживающие первые пять истинными под нагрузкой и под атакой. Иерархия глубиной в одну ступень, призвание, затем конституционные ограничения, затем инженерные инварианты, и конституция гласит, что более глубокого механизма, ими управляющего, не существует.

Таблица 2: Десять ограничений на 1.0.0-rc.7, их ярус и объект, закрепляющий каждое. Таблица удерживается в согласии с кодом проверкой `check_c1c10_objects_resolve`, которая роняет сборку, если названный объект перестал существовать.

#	Ограничение	Ярус	Чем закреплено
C1	След аудита работает только на добавление; история никогда не переписывается и не удаляется	Конституционное	Триггеры отвергают UPDATE и DELETE на каждой таблице аудита (<code>reject_audit_modification</code>)
C2	Проверка с нулевым разглашением не хранит идентификатора токена	Конституционное	Двунаправленный CHECK <code>chk_disclosure_token_consistency</code>
C3	Человек держит не более одного ACTIVE-токена	Конституционное	Частичный уникальный индекс <code>uq_one_active_per_person</code>
C4	Счёт неудачных входов атомарен	Инженерное	Однооператорный UPDATE <code>...RETURNING</code>
C5	Никаких встроенных сценариев; политика безопасности содержимого есть <code>script-src 'self'</code>	Инженерное	Заголовок ответа, проверяемый на каждом маршруте
C6	Уровень раскрытия закрепляется на сервере; клиент не в силах повысить то, что узнаёт	Конституционное	Серверный код в паре с тестами сокрытия; стоящий за ним CHECK C2
C7	Никакой защитой криптографии; алгоритмы суть строки в реестре	Инженерное	Внешний ключ на <code>CryptographicAlgorithm</code>
C8	Всякая карта и всякая сводка API ограничены	Инженерное	Жёсткие потолки в SQL-функциях и маршрутах
C9	Утверждения об одновременности проверяются настоящими потоками	Инженерное	Многопоточные наборы тестов против живой базы данных
C10	Личность не есть деньги; схема не несёт де-нежных полей	Конституционное	Структурное отсутствие, закреплённое проверкой

Репозиторий фиксирует также структурное утверждение о десятке: что набор замкнут, так что одиннадцатое ограничение требует либо заменить одно из имеющихся, либо расширить рамку, и что ограничения зависят друг от друга, так что удаление любого отзывается во всех остальных. Обход зависимостей (уберите C10, и всякое иное ограничение защищает вместо этого систему финансовой слежки; уберите C1, и закрепление всякого иного ограничения становится задним числом отрицаемым) есть довод в пользу обращения с ними как с графом, а не как со списком. Афина, в разделе ??, держит отображение каждого ограничения в точный живой механизм, который его закрепляет, и проверка роняет сборку, если механизм уплывает. У трёх из десяти теперь есть также формальная модель, проверяемая при каждой отправке: покрытие очистки для C1, несвязываемость для C2 через две объеди-

няющие данные доверяющие стороны и СЗ при одновременных выдаче и отзыве, и у каждой есть парная конфигурация, на которой она обязана упасть, так что ни одна из трёх не пуста.

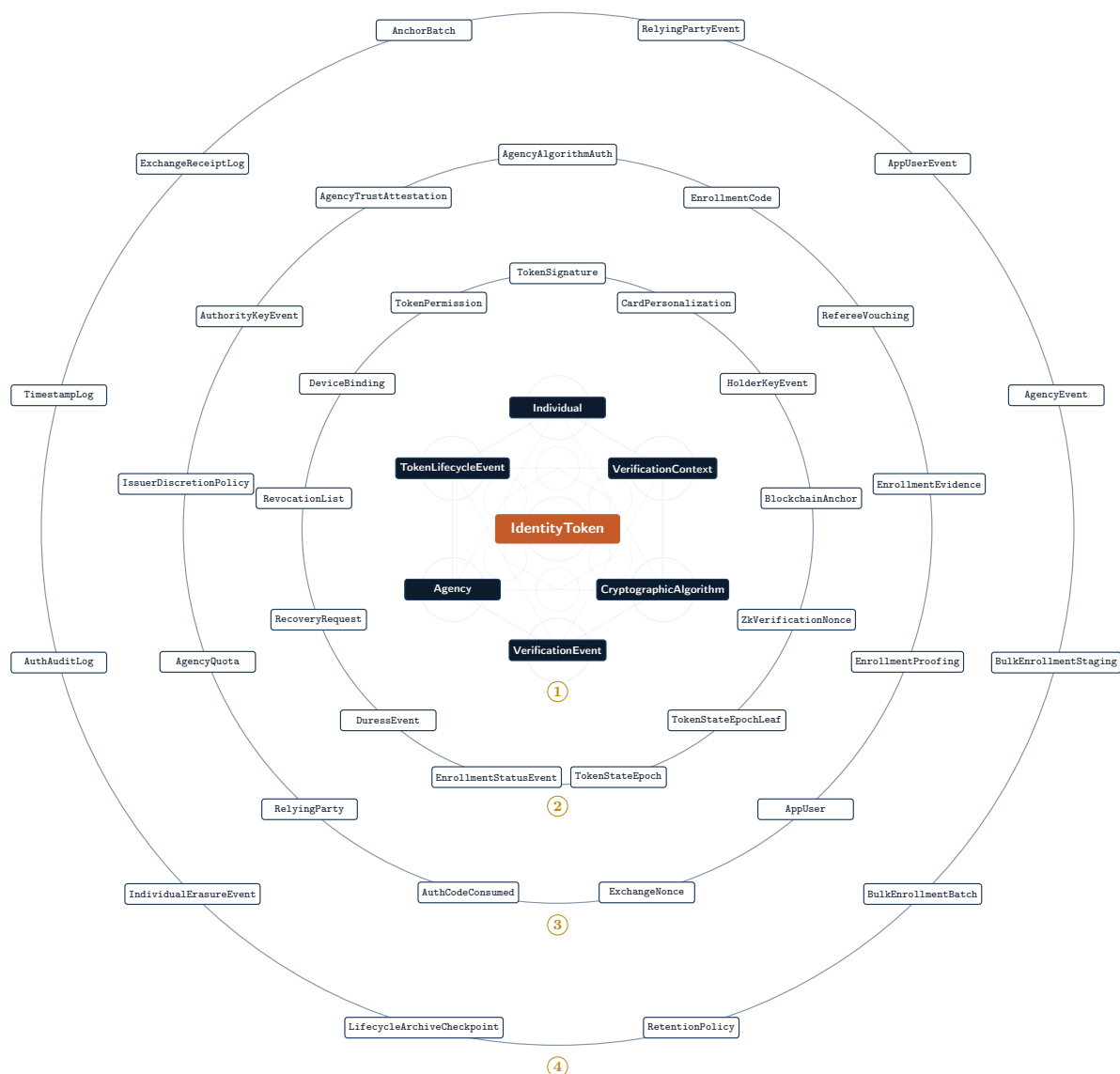


Рис. 4: 45 канонических таблиц `01_schema.sql` на `1.0.0-rc.7` в виде колец вокруг ядра версии 1. Бледная решётка в центре есть геометрия ядра с рис. 2 в тех же пропорциях. Второе кольцо есть механика удостоверения: подписи, разрешения, эпохи, восстановление, принуждение, ключ держателя, карта. Третье кольцо есть полномочия, доверие и люди за стойкой: кто вправе подписывать по какому алгоритму, кто кому доверяет, потолки отзыва, квоты, доверяющие стороны, регистры повторов и то, на чём держалась регистрация. Четвёртое кольцо есть архивы, сторожевые башни и решения, сохранённые как записи: журналы только на добавление, якоря, решение о сроках хранения в виде данных и события, которые записывают всякое изменение органа, операторской учётной записи или доверяющей стороны. В мигрированном развёртывании держится ещё семь таблиц (реестр миграций, операторские сессии и учётные данные, журнал доступа к аудиту и три подготовленные таблицы Афины) и пять дочерних разделов.

4.3 Аудит как сама запись

Четырнадцать таблиц следуют одному названному образцу, аудиту как самой записи: строка и её история суть один и тот же предмет. Рядом с таблицей нет отдельного журнала событий, который можно было бы править, пока таблица заперта. Изменение либо запрещено начисто, либо ограничено одним полем, движущимся в одну сторону, вроде даты устаревания подписи или даты отзыва аттестации. Каждая из четырнадцати закреплена триггером, поднимающим `insufficient_privilege`, и проверка роняет сборку, если хоть один триггер перестанет это делать. Сверх тех четырнадцати, которые называет проектная запись, триггер добавления несёт на 1.0.0-rc.7 тридцать одна таблица, и новые суть записи, которые теперь оставляют за собой решения об органах, операторах, доверяющих сторонах и регистрациях; а начиная с v9.412 C1 проверяется по одному разу на каждую заявляющую его таблицу: управляемый таблицей набор тестов читает триггеры добавления из системного каталога, а не из списка, который кто-то написал, так что новая таблица с добавлением не может просто остаться непроверенной. **РЕАЛИЗОВАНО**

Следствие состоит в том, что ни один внешний ключ нигде в схеме не каскадирует удаление. Каскад позволил бы событиям жизненного цикла удалённого токена исчезнуть вместе с ним. Всякая ссылка по умолчанию есть `NO ACTION`, так что участник, на которого ссылается хоть одна строка аудита, фактически неудаляем, а проверка просматривает каждый SQL-файл на обе формы каскада без единого исключения. Единственная запись, намеренно не несущая никакого внешнего ключа, чтобы прекращение договора с доверяющей стороной не могло стереть её историю, научила репозиторий кое-чему о его собственной тестовой базе данных на 1.0.0-rc.7: посев, перезагружающий образцовый мир, перезапускал идентификаторы сторон с единицы и доходил до этой записи, не имея ни того ни другого, так что первая сторона каждой перезагрузки наследовала события мёртвой стороны. Посев теперь называет эту запись, а проверка вычисляет из собственных внешних ключей схемы, какие записи перезагрузка оставила бы позади.

Роли приложения, `polaris_app`, отозваны `UPDATE` и `DELETE` на таблицах с добавлением, и она не держит привилегии определения данных. Единственный путь, удаляющий строки аудита, есть процедура «сначала в архив, потом очистка», которая сперва сверяет архив с его описью, отвергает всякую отсечку внутри окна хранения и дописывает собственную контрольную строку в режиме добавления. Само окно хранения есть данные с нижним пределом: ни одна строка политики не способна выразить срок хранения короче года, потому что ограничение целостности этого не допускает, а понижение предела есть изменение схемы, за которое отвечают на ревью. Призывание отвергает неограниченное хранение; оно ровно так же отвергает хранение настолько короткое, что оно стирает запись. **РЕАЛИЗОВАНО**

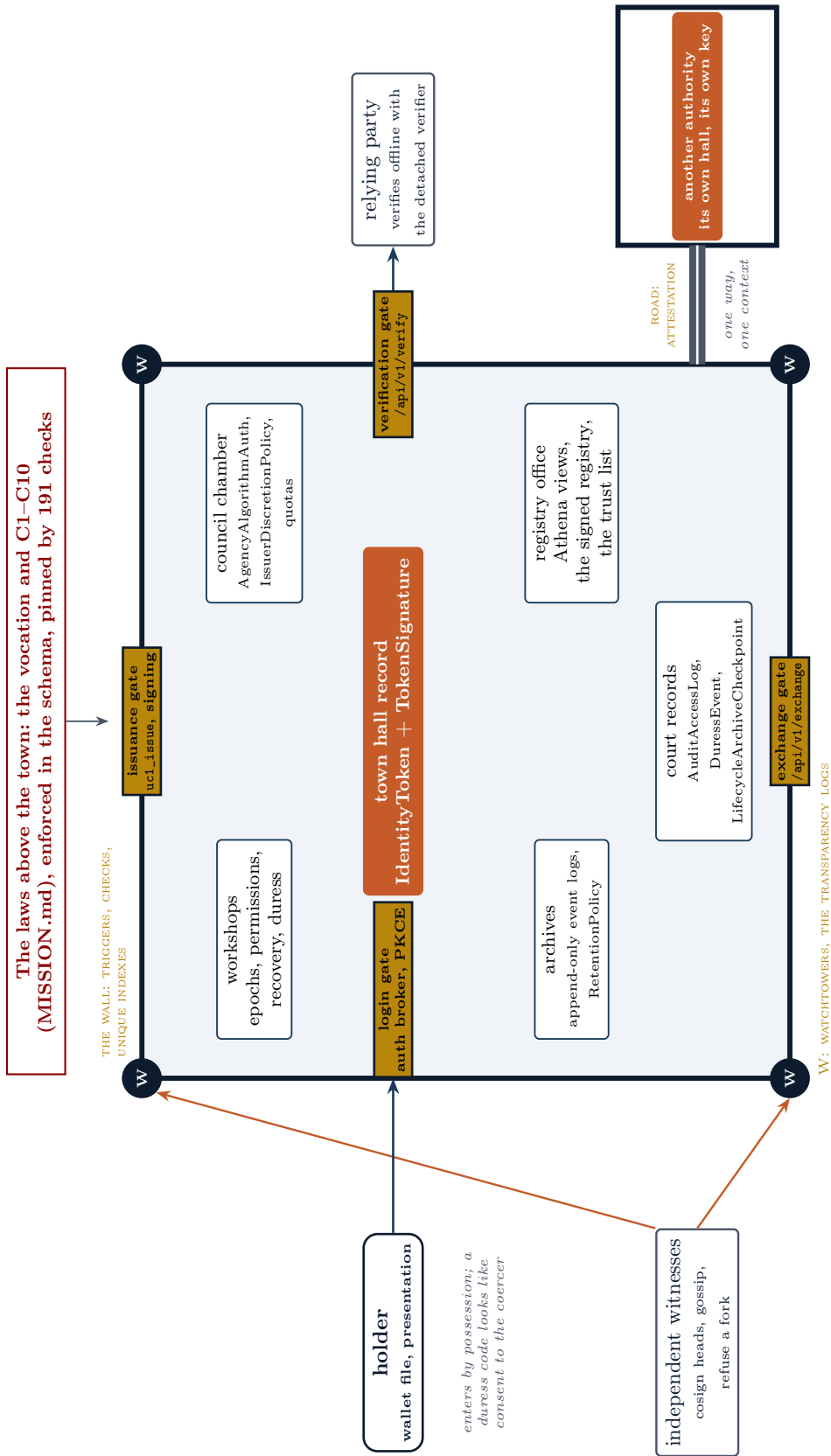


Рис. 5: Обнесённый стеной город, с настоящей составной частью рядом с каждым городским именем. Реестр в ратуше есть ядро удостоверения; стена есть схема; четверо ворот суть маршруты, через которые утверждение входит и выходит; сторожевые башни суть журналы прозрачности, а свидетели за стеной за ними наблюдают; дорога во второй город есть направленная аттестация, привязанная к контексту; законы над городом связывают сам город. План описывает устройство. Polaris федеративен: каждый орган есть свой собственный город.

4.4 Слой инвариантных проверок и учения, которые на него нападают

Над схемой лежит плоский слой из 313 обыкновенных функций, каждая вида `check_*(repo_root)`, которые читают репозиторий и возвращают вывод. Они запирают непрерывную интеграцию: любой отказ роняет сборку. Каждая проверка сопряжена с тестом обнаружения, который ломает образец ровно тем способом, какой проверка обещает поймать, и утверждает, что проверка краснеет; а начиная с v9.405 он утверждает ещё и то, что проверка проходит на том же образце в целом виде, потому что тест, который только и делает, что утверждает отказ, не установил, что он вообще что-нибудь обнаруживает. Проверка, не способная обнаружить собственное нарушение, считается сломанной. Это и есть механизм, которым утверждения прозы обращаются в запоры: величина, названная в README, маршрут, обязанный вернуть заголовок, миграция, обязанная не каскадировать, учение, которое CI обязан прогнать. **РЕАЛИЗОВАНО**

Версия 2 описала проверки и на этом остановилась. Между v9.398 и v9.437 репозиторий задал этому слою тот вопрос, какой задаёт всему остальному, не *проходит ли он*, а *замечил бы ли он*, и ответом часто было нет. Закомментирование строк, несущих искомые строки проверки, что выглядит ровно как временно отключённая строка, оставило зелёными 71 проверку из 75, потому что вспомогательная функция, через которую читает каждая проверка, выдавала ей комментарии вместе с кодом. Удаление каждого из 46 ограничений СНЕСК показало, что все до единого несущие, и это была хорошая новость; удаление каждого из 37 триггеров нашло 14, которых никто не заметил, и двенадцать из них были стражами добавления, которыми C1 *и является*; удаление каждого из 17 уникальных индексов нашло двенадцать непокрытых и остановилось на втором, потому что в отсутствие `uq_one_active_per_person` набор тестов, существующий ради доказательства C3, успел вставить и зафиксировать второй действующий токен для одного человека, и индекс уже нельзя было перестроить. Удаление каждого из 59 отказов, которые делают шестнадцать хранимых процедур, нашло 27 таких, что ни один тест по ним не затосковал бы, и сгущены они были ровно там, куда триггер не видит: правило четырёх глаз, период выдержки, три канала и третий свидетель обряда восстановления, предусловия необратимого стирания. Каждая из этих вещей теперь покрыта, и каждое из этих измерений теперь есть учение, выполняемое при каждой отправке с отрицательным контролем, так что результат «ноль выживших» отличим от стенда, который не прогнал ничего. На 1.0.0-rc.7 таких учений двенадцать: над проверками, ограничениями, триггерами, процедурами, свидетельствами с нулевым разглашением, контрактом соответствия, двумя эталонными верификаторами, верификатором OpenID4VP, приложением, конституционными ограничениями, поверхностями, по которым квантифицирует проверка, и перемещениями кода между модулями. Раздел ?? несёт эту запись. **РЕАЛИЗОВАНО**

Карточка слоя: стены

Что это	PostgreSQL 16: 45 таблиц, 42 триггера, шестнадцать хранимых процедур, реализующих каждый сценарий со сменой состояния, таблицы аудита только на добавление, политика хранения в виде данных.
Зачем оно существует	Обещание, закреплённое лишь людьми, которые эксплуатируют систему, обещанием не является. Схема связывает каждого клиента, каждое восстановление и каждого инсайдера с подключением к базе данных, кроме владельца.
Чему доверяет	Движку базы данных и его владельцу; правильности ограничений в том виде, как они написаны, ради чего каждое теперь и проверяется мутацией.
Чему отказывает	Коду приложения, процессу ревью и дисциплине оператора как последней линии: приложение никогда не закрепляет конституционную гарантию само по себе. Тесту, которому ни разу не показали, как он падает.
Что видит	Всё, что система хранит. Поэтому слой приватности ограничивает то, что хранится, а не то, кому дозволено читать.
Чего не видит	Биометрические шаблоны, ключи на стороне держателя, регистрационные документы, содержимое обмениваемых сообщений или документов с отметкой времени: ничто из этого в базу данных не попадает.
Если откажет	Убранный триггер, пониженный предел хранения или каскад роняют сборку через проверки; ограничение, утраты которого набор тестов не заметил бы, роняет мутационное учение; у владельца базы данных с консолью возможностей больше, чем у любого триггера, о чём модель угроз говорит, а не умалчивает.
Кем проверяется	check_aor_append_only_triggers, check_aor_privilege_boundary, check_no_fk_cascade, check_retention_engine, check_append_only_tables_are_tested_exhaustively, check_constraint_suite_is_mutation_tested, check_triggers_are_mutation_tested, check_unique_rules_are_tested_exhaustively, check_procedure_refusals_are_mutation_tested, check_seed_restart_resets_dependent_records, самопроверки SQL.
Сейчас или потом	Сейчас.
Внутри и снаружи	Внутри: ядро удостоверения. Снаружи: криптографические свидетельства, потому что ограничение связывает лишь ту базу данных, которая его держит.

Ограничение связывает базу данных, а не копию утверждения, эту базу покинувшую. В тот миг, когда удостоверение передано постороннему или историю органа читает аудитор, ничто из схемы с ними не едет. Следующий слой заставляет утверждения везти доказательство с собой.

5 Удостоверение истины: подписи, хранение ключей, два свидетеля и одна миграция

5.1 Для чего нужна подпись

Цифровая подпись есть печать, поставить которую способен лишь владелец закрытого ключа и проверить которую способен всякий, у кого есть парный открытый ключ. Polaris подписывает дайджест SHA3-256 значения токена ключом ML-DSA выдающего ведомства и хранит подпись рядом с открытым ключом, который её поставил. Утверждение, которое несёт подпись, узко и постоянно: *этот ключ подписал эти байты*. Оно ничего не говорит о том, сохраняются ли у токена полномочия, что есть задача следующего раздела, и ничего о том, кто владеет ключом, что есть задача хранения. **РЕАЛИЗОВАНО**

Алгоритм по умолчанию есть ML-DSA-65 (FIPS 204, категория стойкости 3), а рядом с ним ML-DSA-87 (категория 5); ML-DSA-44 отвергается повсюду как лежащий ниже предела. Ни одно подписанное тело не зашивает в себя свой алгоритм: хранимый ключ решает это перед подписанием, потому что `algorithm` сам есть подписываемое поле, а реестр объявляет принимаемый набор. SLH-DSA-128s и SLH-DSA-256s, хеш-основанные запасные варианты, не опирающиеся на решёточные допущения, зарегистрированы в таблице алгоритмов именно потому, что опираются на одни лишь хеш-функции, так что поворот прочь от решёток есть

обновление строки; ни один подписант SLH-DSA не подключён, и книга учёта состояния несёт этот пробел. **РЕАЛИЗОВАНО** для ML-DSA; **ПРЕДЛОЖЕНО** для хеш-основанного подписанта.

5.2 Хранение ключей

Закрытый ключ никогда не лежит в коде приложения. Он лежит за интерфейсом хранения с тремя драйверами: файл (разработка), PKCS#11 (аппаратный модуль безопасности или программный токен; ключ вырабатывается внутри токена и наружу не выходит) и облачная служба управления ключами. Непрерывная интеграция прогоняет драйвер PKCS#11 против программного токена, реализующего ML-DSA, а не против аппаратного модуля, и проворачивает внутри него замену ключа: токен, подписанный старым внутритокенным ключом, по-прежнему проверяется после переключения, а новый ключ уже подписывает. Конвертная криптография драйвера KMS настоящая, а его служба есть заместитель. Один промышленный профиль делает модуль единственным подписантом: приложение отказывается запускаться, если присутствует файловый ключ или настоящее подписание недоступно. Какой драйвер, какой модуль и кто держит ключ, суть решения, которые книга готовности возлагает на развёртывающую организацию. **РЕАЛИЗОВАНО** для драйверов и учения; **ТРЕБУЕТСЯ ВНЕШНЯЯ ПРОВЕРКА** для аппаратного хранения, поскольку ни один аппаратный модуль не применялся.

Заглушка заслуживает отдельного абзаца, потому что именно здесь честную систему легче всего прочесть неверно, и версия 3 говорит об этом прямее, чем говорила версия 2. Без флага настоящего подписания и без установленной решёточной библиотеки подписывающий модуль производит детерминированную привязку SHA3-256 с пометкой DETERMINISTIC-PLACEHOLDER-SHA3-256: тридцать два байта, не проверяемые ни против какого ключа, тогда как настоящая подпись ML-DSA-65 занимает 3309. Это и есть значение по умолчанию при разработке и на большей части непрерывной интеграции. Оно ограждено, начиная с v9.280 промышленный запуск падает закрытым, если настоящее подписание действительно недоступно, заглушка есть именованный профиль разработки, и она громко предупреждает, когда её применяют безымянно; и всё же именно из-за неё репозиторий больше не называет себя постквантовым, не оговорив этого тут же. История записана, а не спрятана: на v9.58 заглавное утверждение о постквантовости было, на уровне данных, зашитой строкой, а настоящий подписывающий модуль оставался необитаемым островом, пока его не подключили к выдаче. **РЕАЛИЗОВАНО**

5.3 Два свидетеля при выдаче, один при использовании с выборочной проверкой и один внешний набор

Принцип двух свидетелей гласит, что всякий криптографический вердикт, который Polaris выпускает наружу, обязан быть проверяем второй, независимой реализацией, написанной отдельно, представленной иначе, не разделяющей ни строки кода и либо соглашающейся с вердиктом, либо явно воздерживающейся. Верификатор, согласный сам с собой, не установил ничего. При выдаче только что произведённая подпись проверяется двумя независимыми реализациями FIPS 204, решёточной библиотекой и библиотекой на основе OpenSSL, и ей отказывают в сохранении, если её не принимают обе; отсутствующий свидетель при выдаче есть отказ, а не понижение. **РЕАЛИЗОВАНО**

Проверка при использовании прогоняет одного свидетеля, потому что про сохранённую подпись уже известно, что она проверяется обоими, а один свидетель обнаруживает подмену примерно вдесятеро быстрее: на эталонной машине около 7800 проверок в секунду на ядро против примерно 740 при двух свидетелях. Быстрый путь зарабатывает свою скорость тем, что его непрерывно проверяют: обязательная случайная доля успешных проверок переигры-

ается вторым свидетелем, расхождение увеличивает счётчик и вызывает пейджер с наивысшей важностью, а каждый ответ называет, какой набор свидетелей отработал. **РЕАЛИЗОВАНО**

Одна вещь про свидетелей не замкнута на себя. При каждой отправке векторы проверки ML-DSA-65 из Project Wycherproof, сторонний набор, привязанный к коммиту, в котором принятие подписи, названной набором недействительной, есть отказ, прогоняются под обоими свидетелями: 58 из 58 совпадают. Это свидетельство того, что проверка примитива в Polaris сходится с внешним авторитетом на 58 случаях. Это не кошелёк, не результат по совместимости, и оно ничего не говорит о протоколе удостоверений; сводная таблица репозитория записывает это ровно этими словами. **ПРОВЕРЕНО ИЗВНЕ** для примитива.

Тот же принцип управляет вердиктом с нулевым разглашением (раздел ??): повторная реализация на Python с обыкновенными целыми числами по модулю простого, не разделяющая ни строки кода с доказывателем на Rust, заново выводит утверждение о принадлежности и обязана согласиться, а воздерживается она открыто по той единственной оси, которую моделировать не в силах, по целостности байтов доказательства. Что принцип ловит, так это расхождение реализаций. Чего он не ловит, так это общего неверного прочтения спецификации, и репозиторий говорит об этом рядом с каждым утверждением о двух свидетелях. Начиная с v9.419 свидетели также проверяются мутацией: утверждение, что две реализации обязаны согласиться, есть теперь утверждение и о тестах, а не только о коде.

5.4 Каноническое подписание: одни и те же байты с обеих сторон

Всякий подписанный предмет в Polaris есть JSON-утверждение с упорядоченными ключами и без пробельных символов, взятое дайджестом SHA3-256 и затем подписанное. Приложение строит утверждение; отделяемый верификатор (раздел 6) перестраивает его из предмета, чтобы проверить подпись. Два построения обязаны совпадать побайтово, иначе подлинный предмет проверку не пройдёт, и тест, оракул канонической равнозначности, закрепляет каждый подписываемый тип с обеих сторон. Единственная ошибка, ради которой оракул и существует, была найдена на ревью: предмет, поле `algorithm` которого выставлялось после подписания, нёс подписываемое поле, не покрытое подписью. **РЕАЛИЗОВАНО**

5.5 У ключей есть жизненный цикл

Реестр ключей есть таблица событий, работающая только на добавление: зарегистрирован, выведен из обращения или скомпрометирован с момента, который может предшествовать обнаружению, и из неё публикуется подписанный список доверия. Верификатор, держащий список доверия, определяет состояние ключа на любой момент, так что удостоверение под скомпрометированным ключом эмитента отвергается независимо от собственной описи эмитента, которую вор, завладевший ключом, мог бы подделать, а документ, подписанный и помеченный временем до компрометации, остаётся действительным, тогда как подписанный после не остаётся. Учение на двух экземплярах записывает компрометацию в одном органе и наблюдает, как решение другого переворачивается по проводу, без всякого содействия скомпрометированной стороны. **РЕАЛИЗОВАНО**

5.6 Гибкость при проверяемом пути миграции и во что обошёлся бы взлом

Этот подраздел новый, и он заменяет собой одно слово. Версия 2 называла Polaris постквантовым на обложке. Внешний довод, что фраза пуста, поскольку никто не знает, каких размеров достигнут квантовые вычислители, нападал не на то, но то, в чём он был прав, острее собственной подачи: стойкость ML-DSA-65 не опирается на предсказание о железе, она опирается на трудность Module-LWE и Module-SIS против лучших известных классических и квантовых алгоритмов, а это может оказаться неверным по математическим причинам. Два

настоящих риска суть криптоаналитический прорыв против решёточных допущений и изъясн реализации, и устройство уже отвечает на оба, хеш-основанным подстрахованием в реестре и двумя свидетелями при выдаче. Так что задачей была не инженерия; задачей была формулировка. Репозиторий способен показать *алгоритмическую гибкость при проверяемом пути миграции*: C7 делает алгоритм полем, `AgencyAlgorithmAuth` разрешает его для каждого органа, а население можно переподписать под новым набором параметров путём, выполняемым при каждой отправке. **РЕАЛИЗОВАНО**

Этот путь измерен, а не запланирован. `polaris migrate-population` переподписывает всё действующее население под новым набором параметров и построен из тех же ограничений, на которые опирается потоковая процедура: уникальное ограничение есть точка упорядочивания, так что два исполнителя, состязаящиеся за одно удостоверение, производят одну строку, а шестьдесят четыре исполнителя делят население без всякого согласования. Значение, которое важно, есть не пропускная способность, а то, сколько держателей держат удостоверение, не проверяемое ничем, и учение берёт эту выборку на каждой границе пакета, а не по концам, потому что разрыв, который открывается и закрывается между двумя конечными точками, невидим для проверки «до и после». Он равен нулю на всём протяжении. Старая подпись продолжает проверяться до своей даты устаревания, и этот промежуток *и есть* миграция: объявить вытесненный алгоритм устаревшим отказываются, пока хоть одно действующее удостоверение не мигрировало, потому что иначе держатель обнаружил бы разрыв на границе, пока консоль сообщала бы об успехах. Под настоящим ML-DSA-87 с одним хранимым ключом учение переподписывает около 345 удостоверений в секунду на одном исполнителе, и 91 процент этого времени занимает подписание, так что население в 350 миллионов есть около двенадцати дней на одном исполнителе и четыре с половиной часа на шестидесяти четырёх; закупка мощности базы данных под это почти ничего не даёт. **РЕАЛИЗОВАНО**

Во что обошёлся бы взлом Module-LWE, записано в книге готовности, и настоящий документ это повторяет. Он не потребовал бы изменения схемы, изменения на пути проверки или новой модели доверия. Он стоил бы всякого удостоверения, уже выданного под взломанным алгоритмом, и ещё одной вещи, которой никакая гибкость не возвращает: во время переключения удостоверение несёт классическую подпись рядом с постквантовой, а классическую половину не защищает ничто из того, что даёт это устройство, так что удостоверение, собранное сегодня, станет читаемым в день падения его классической подписи, на что бы система ни мигрировала потом. Эта несимметричность и есть причина, по которой подписывать постквантово стоит уже сейчас, и она же есть предел того, чего этим достигают.

Собственная лаборатория репозитория затем спросила, на той ли стороне находится гибкость, которой она нужна, и обнаружила, что гибкость есть только со стороны эмитента. Поле `deprecation_date` алгоритма не доходит ни до одного подписанного предмета: реестр публикует голый перечень имён алгоритмов, список доверия несёт состояние только по каждому ключу, а принимаемый набор отделяемого верификатора есть словарь в коде. Поэтому верификатор не способен узнать, что алгоритм объявлен устаревшим, вывод алгоритма из обращения есть выпуск программного обновления для каждой доверяющей стороны, а не миграция, и отзыв ключа этого не заменяет, потому что если допущение рухнет, нападающий подделает подпись под любым ключом, который список доверия называет действующим. Откат и смешанное окно во время частичной миграции не измерены. Изменений не предлагается; находка записана как ограничение для развёртывающей организации. **ТРЕБУЕТСЯ ВНЕШНЯЯ ПРОВЕРКА**

5.7 Что остаётся классическим

Подпись токена, дайджесты, обязательства Меркла, доказательство с нулевым разглашением, обмен ключами от клиента до кромки для современных клиентов и, начиная с v9.404, переход от приложения к пулеру соединений суть постквантовые, причём последние два считаны с настоящего рукопожатия, а не выведены из базового образа, из-за чего книга учёта состояния когда-то назвала один из них классическим, тогда как он таким не был. Переход от пулера к базе данных, подписи сертификатов и учётные данные WebAuthn оператора остаются классическими, каждое из этого упирается в третью сторону (выпуск PostgreSQL, публичная экосистема сертификатов, железо аутентификаторов) и каждое изложено в книге учёта состояния со своей угрозой и своей открытостью. Ни решётчатая библиотека, ни система доказательств, ни схема не проходили внешнего криптографического аудита. **ТРЕБУЕТСЯ ВНЕШНЯЯ ПРОВЕРКА**

Карточка слоя: криптографические свидетельства

Что это	Подписи ML-DSA над дайджестами SHA3-256, хранимые вместе со своими открытыми ключами; интерфейс хранения с драйверами файла, PKCS#11 и KMS; две независимые проверяющие реализации и один внешний набор векторов; канонический формат утверждения; реестр ключей только на добавление и подписанный список доверия; работающая миграция населения.
Зачем оно существует	Утверждению, покидающему базу данных, нужно везти доказательство с собой; доказательство, проверить которое способна лишь одна программа, есть обещание; а алгоритм, который нельзя заменить, есть назначенный срок.
Чему доверяет	FIPS 204 и 202 в том виде, как они стандартизованы; тому, что две библиотеки не разделяют одной ошибки; той подсистеме хранения, которую выбрал оператор.
Чему отказывает	Вердикту одной реализации при выдаче; алгоритму, названному в коде, а не ключом; подписи, сохранённой без своего ключа; заглушке, выдающей себя за подпись; объявлению устаревания, пока хоть одно удостоверение осталось бы не проверяемым ничем.
Что видит	Байты, которые подписывает, а они суть дайджесты: путь подписания никогда не видит человека.
Чего не видит	Сохраняются ли у подписанного утверждения полномочия (раздел 7); знает ли верификатор в поле, что алгоритм выведен из обращения.
Если откажет	Поддельная подпись падает у обоих свидетелей; скомпрометированный ключ выводится из обращения с указанного момента через реестр, и всякий верификатор, держащий список доверия, отвергает подписанное им после; взломанный алгоритм стоит каждого удостоверения под ним и переживается переподписанием, со измеренной скоростью.
Кем проверяется	check_pqc_signing_wired, check_pqc_second_witness, check_verify_witness_sampling, check_real_pqc_default_boot, check_trust_lifecycle, check_algorithm_agility, check_quantum_event_readiness, check_zk_witnesses_are_mutation_tested, check_post_quantum_claims_are_agility, оракул канонической равнозначности, набор атак.
Сейчас или потом	Сейчас. Аппаратный модуль и внешний аудит потом, и принадлежат они развёртывающей организации; подписанное состояние алгоритма, которое мог бы прочесть верификатор, не построено.
Внутри и снаружи	Внутри: схема, хранящая строки подписей. Снаружи: независимые верификаторы, проверяющие их без эмитента.

Подпись, которую проверяет собственный сервер эмитента, не доказывает постороннему ничего, потому что постороннему приходится доверять серверу. Следующий слой забирает проверку из рук эмитента целиком и, впервые, отдаёт её в руки программ, написанных не автором эмитента.

6 Проверка без эмитента: пакетные верификаторы, контракт соответствия и первые посторонние

6.1 Отделяемый верификатор как вещь, которую устанавливает посторонний

`polaris-verify` проверяет пакет подлинности удостоверения одной лишь стандартной библиотекой ML-DSA: ни строки кода Polaris, ни базы данных, ни сети, если только его не попросили запросить состояние. Версия 2 описывала его как сценарий в репозитории. Теперь это пакет в публичном реестре, на `1.0.0-rc.3`, кандидат в выпуски, под которым с 15 сентября 2026 года лежит `0.1.0`, опубликованный собственным рабочим процессом репозитория по доверенной публикации, так что нигде не существует долгоживущих учётных данных, а сценарий, который называла версия 2, есть переходник на него. Он отказывается запускаться, пока запуск не скажет, какую криптографию выполняет: `-pqc-provider oqs, cryptography` или `auto`, либо `-dev-placeholder`, без значения по умолчанию и без переменной окружения, потому что верификатор, тихо переставший проверять, всё равно отвечает на каждый вопрос. Всякий машиночитаемый вердикт несёт поле `crypto`, а под флагом разработки настоящие подсистемы выключаются, так что запуск для разработки вообще не способен произвести истинный вердикт; построение этого запора обнаружило, что первая версия флага объявляла режим, в него не входя, а это ярлык, а не режим. **РЕАЛИЗОВАНО**

Это единственная составная часть Polaris, вход которой выбирает противник, потому что доверяющая сторона запускает её на том, что предъявил посторонний, и это та часть, через которую проверяется всякий последующий предмет протокола: описи, контрольные точки эпох, ленты отзывов, наборы состояний, утверждения о состоянии, вершины прозрачности, расписки, отметки времени, подписанные документы, реестры, списки доверия, предъявления, доказательства держателя, полномочия агентов. Три независимые реализации ML-DSA сходятся на опубликованных векторах: решётчатая библиотека, OpenSSL и третья в TypeScript-SDK. *Тотальность* против враждебного входа удерживается метаморфическим фаззером, а рядом с ним работает набор атак из исполняемых противников; ггер доказывает, что строка существует, атака доказывает, что защита держит. **РЕАЛИЗОВАНО**

Способен ли посторонний в действительности его установить, измеряется, а не предполагается. При каждой отправке и перед каждой публикацией учение собирает колесо, ставит его в одноразовое окружение, где нет ничего от Polaris, и запускает из каталога, который не есть репозиторий, со снятыми переменными `POLARIS_*`: запретная исполняемая поверхность (веб-приложение, драйвер базы данных, слой проверок) обязана отсутствовать, команда обязана отказаться запускаться без объявленного режима криптографии, подлинное удостоверение ML-DSA-87 обязано провериться, а его подменённый двойник обязан не провериться. Учение несёт отрицательный контроль, второе колесо с прикрученным к верификатору импортом базы данных, который стенд обязан поймать. Это важно, потому что всякий иной набор тестов в репозитории работает изнутри репозитория с репозиторием на пути поиска; импорт, который остался бы там зелёным, падает на машине первого же постороннего. **РЕАЛИЗОВАНО**

6.2 SDK и контракт соответствия

Доверяющая сторона ставит один из двух проверочных SDK, `polaris-sdk-python` и `polaris-sdk-ts`, каждый из которых отвечает на один узкий вопрос о предъявленном удостоверении, подлинно ли оно и полномерно ли прямо сейчас, и никогда не возвращает данных человека. Оба лежат в своих реестрах. Тот, что на TypeScript, есть урок о том, чему учит реестр: его первая упаковка направляла `exports` на исходный код TypeScript, который Node внутри `node_modules` обрабатывать отказывается, так что пакет не смог бы импортировать ни один потребитель ни на одной версии Node; тесты репозитория работали внутри ката-

лога пакета, где обработка разрешена, и ничего не заметили. Теперь он компилируется в дистрибутив, а учение ставит упакованный архив в пустой проект. **РЕАЛИЗОВАНО**

Оба удерживаются языконезависимым набором проверок соответствия: исполнитель гоняет любой верификатор через стандартный ввод и вывод по опубликованным случаям, коих на 1.0.0-rc.7 насчитывается 118 по двадцати пяти типам предметов, включая составное решение о доверии в федерации, доказательство держателя, полномочие агента и его отзыв. Прохождение этого набора и означает *соответствие*, а нормативная спецификация формата передачи (язык RFC 2119, двадцать семь форматных строк, точный перечень подписываемых полей каждого предмета) есть то, против чего строила бы реализация, не импортирующая ни строки кода Polaris. **РЕАЛИЗОВАНО**

Затем у контракта спросили, чего он не спрашивает, измерением, и ответ ограничивает то, что означает соответствие. У трёх типов предметов не было ни одного случая с испорченной подписью, так что верификатор, ни разу их подпись не проверявший, соответствовал. Пять из опубликованных действительных векторов истекли месяцами раньше, и ни один случай этого не заметил, потому что ни один не утверждал свежести. Вопрос о свежести вскрыл два расхождения между поставляемыми эталонными верификаторами: ни один не ограничивал возраст доказательства держателя, так что интегратор, следовавший за одним из них, не получал защиты от повторов при предъявлении, и ни один не умел сообщить, доверенный ли у предмета эмитент вообще, так что подлинная подпись постороннего проходила. Всё исправлено, и три верификатора сходятся на каждом случае. Чего контракт по-прежнему не доказывает, изложено там, где интегратор это прочтёт: 43 поля вердикта не стеснены ни одним опубликованным случаем, и ни одно из них нельзя закрыть, написав такой случай, потому что каждому нужен соответствующий верификатор, вычисляющий то, чего он не вычисляет. Контракт стесняет то, что вычисляет его слабейшая соответствующая реализация. Затем мутировали и сами эталонные верификаторы: каждый отказ в каждом из них поочерёдно обращали, и восемнадцать из восемнадцати в реализации на Python и девять из четырнадцати в реализации на TypeScript приняли то, ради отклонения чего они и существуют, при зелёных тестах самого SDK и зелёном исполнителе соответствия, включая исчерпанный счётчик использований в полномочии агента и, в TypeScript-SDK, сравнение за постоянное время, у которого обращённая проверка длины заставила пятибайтовое значение сравниться с 32-байтовым корнем. Всё это закрыто в собственных тестах SDK, а учение обращает все 32 отказа при каждой отправке с отдельным отрицательным контролем на каждый SDK. **РЕАЛИЗОВАНО**

6.3 Версионирование и замороженная версия 1

Старшие версии протокола живут в форматной строке (`polaris-registry/1`), младшие объявляются реестром и для проверки никогда не нужны, а правило согласования нормативно: отвергни неизвестную старшую, прими любую младшую, никогда не выпускай необъявленной версии, ответь на неподдерживаемую определённой ошибкой и списком поддерживаемых. Версия 1 протокола заморожена под контрольными суммами, рядом с ней помещён отделяемый верификатор `v9.317`, а набор тестов на совместимость работает в обе стороны при каждой отправке: сегодняшние верификаторы обязаны принимать всё, что опубликовала версия 1 (44 случая из 44 держатся), а закреплённый старый верификатор обязан никогда не принимать того, что сегодняшний набор отвергает. **РЕАЛИЗОВАНО**

6.4 Снаружи: верификатор для кошельков, которых Polaris никогда не встречал

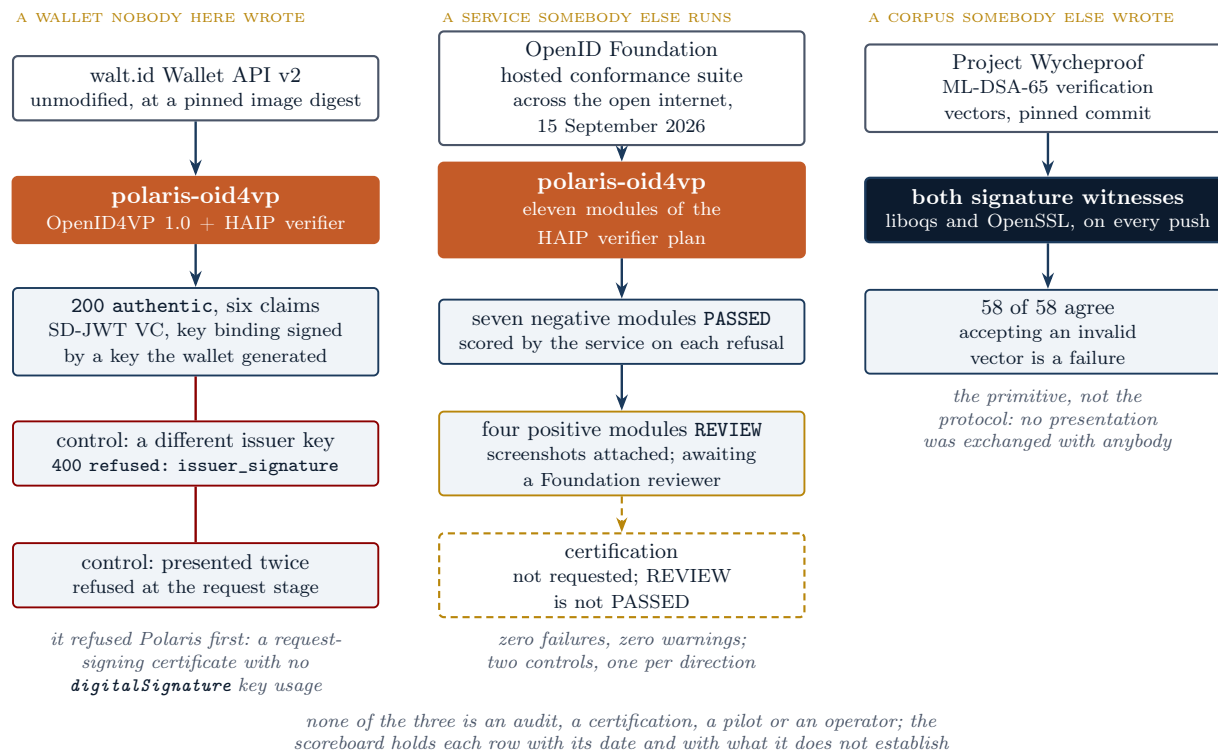


Рис. 6: Три вещи за пределами репозитория, которые его задействовали, и два контроля, без которых первая из них ничего бы не значила. Кошелёк, написанный здесь никем, предъявил удостоверение и был принят; тот же кошелёк, когда верификатор доверял другому ключу эмитента, получил отказ; и повторное предъявление против уже отвеченного запроса получило отказ. Размещённый пакет Фонда сам засчитал семь отказов и оставил четыре принятия на человека-рецензента. Опубликованный набор векторов сходится с обоими свидетелями подписи на примитиве. Ни одна из трёх не есть аудит, сертификация или пилотное внедрение.

Всё вышеизложенное есть Polaris, проверяющий Polaris, и честной границей версии 2 было то, что ни одна внешняя реализация с ним не взаимодействовала. Теперь это предложение иное, на одном пути, а путь был выбран потому, что на нём в действительности и говорит сторонний кошелёк. По операционному договору первой целью по совместимости был верификатор OpenID4VP 1.0 по профилю высокой надёжности взаимодействия, а у пакета проверок соответствия OpenID Foundation есть план тестов ровно для этой роли, `oid4vp-1final-verifier-haip-test-plan`, в котором *удостоверение подписывает сам пакет*, так что собственная выдача Polaris под ML-DSA в этом не участвует. `polaris-oid4vp` есть этот верификатор: он проверяет SD-JWT VC, подписанный ES256, с привязкой к ключу держателя, доставленный как зашифрованный ответ `direct_post.jwt`, и отказывает с названной причиной, по одной на каждый отрицательный модуль плана: `issuer_signature`, `sd_hash`, `kb_signature`, `nonce`, `audience`, `kb_freshness`. Это отдельный пакет, потому что `polaris-verify` обещает не открывать ни одного сокета, а этому слушать необходимо. Одна из его заготовок не замкнута на себя: полный ответ, произведённый собственным кошельком пакета проверок и построенный Java-библиотекой, которой этот проект не пользуется, помещён в дерево, расшифровывается и проверяется, а годом позже получает отказ под другим одноразовым числом, под другой аудиторией и с одним переписанным раскрытием. **РЕАЛИЗОВАНО**

Затем произошли две вещи, которых репозиторий не делал сам с собой. 15 сентября 2026 года размещённая служба проверки соответствия OpenID Foundation прогнала все одина-

дцать модулей плана против верификатора через открытый интернет, забирая объект запроса и отправляя ответ снаружи: ноль отказов, ноль предупреждений. Семь отрицательных модулей, в которых кошелек пакета посылает предъявление, сломанное одним определённым образом, несут собственный `result: PASSED` службы, засчитанный автоматически по отказу верификатора, что делает их самым близким к внешнему противнику, что этому проекту довелось получить. Четыре положительных модуля завершились в состоянии `REVIEW`, которое пакет отводит человеку, чтобы тот удостоверил, что верификатор отобразил успешную проверку; снимки экрана приложены, а рецензент Фонда видит их лишь при подаче на сертификацию, которая не запрашивалась. `REVIEW` не есть `PASSED`, и Polaris не сертифицирован. Первое описание прогона говорило: одиннадцать из одиннадцати чисто, и это было ложью в лестную сторону: для каждого модуля использовался один закреплённый псевдоним, а пакет прерывает тест, когда под тем же псевдонимом начинается другой, и четыре положительных были убиты своими преемниками, что пакет уже записал внутри того самого журнала, который скачали ради подтверждения утверждения. Прогон повторили по одному модулю за раз. Прогон несёт два отрицательных контроля, по одному на направление: верификатор, пропатченный отказывать во всём, замечается положительными модулями, а пропатченный принимать всё, всеми семью отрицательными. [\[ПРОВЕРЕНО ИЗВНЕ\]](#) для профиля; [\[ТРЕБУЕТСЯ ВНЕШНЯЯ ПРОВЕРКА\]](#) для сертификации.

И в тот же день неизменённый `walt.id Wallet API v2`, опубликованный образ по закреплённому дайджесту, предъявил `SD-JWT VC` верификатору по `OpenID4VP` и был принят: он забрал подписанный объект запроса, сверил его с зарегистрированным якорем доверия, сопоставил удостоверение с запросом, подписал токен привязки к ключу ключом `P-256`, который выработал сам и которым этот репозиторий никогда не владел, зашифровал ответ, а `polaris-oid4vp` проверил всю цепочку. Сперва он Polaris отверг, и был прав. Собственный генератор ключей верификатора произвёл сертификат для подписи запросов вовсе без расширения назначения ключа, так что сертификат, вся работа которого состоит в подписании объектов запроса, не был помечен как пригодный для подписи; одиннадцать модулей соответствия, 143 теста пакета, мутационное учение по каждому отказу и каждая проверка в слое инвариантов прошли мимо этого, а первая же реализация извне отказалась продолжать. Исправлено в тот же день, с тестом, который краснеет на листе, где расширения нет, и засчитано в сводной таблице как единственное исправление, вызванное внешней находкой. Обмен держится на тех же двух контролях, потому что верификатор, принимающий всё, печатает ту же самую строку об успехе. Чего он не устанавливает, изложено там же: один кошелек, один формат удостоверения, один путь предъявления, `ES256` повсюду, что и закрепляет профиль; ничего о любом ином кошельке, об `ISO 18013-5` или о постквантовом пути, и никаких утверждений о том, что верификатор совместим вообще. Всякий может повторить этот обмен с чистой машины примерно за десять минут, не клонируя репозитория; страницу, объясняющую как, исполнили прежде, чем написали. [\[ПРОВЕРЕНО ИЗВНЕ\]](#) для этого пути.

Родной протокол, форматы `polaris-*/1`, на которых построено всё остальное в настоящем документе, остаётся там, где его оставила версия 2: спецификация, случаи и два SDK существуют, а реализации сторонней командой по одним лишь документам не случилось. [\[ТРЕБУЕТСЯ ВНЕШНЯЯ ПРОВЕРКА\]](#)

Два формата-моста делают границу явной. Удостоверение Polaris отображается в структуру мобильного документа `ISO/IEC 18013-5`, и считыватель, говорящий на этом стандарте, его разбирает, обходит его пространства имён и сверяет дайджест каждого раскрытого элемента с подписанным объектом безопасности, причём доказывает это независимая реализация `CBOR`, а не та, что писала байты; подпись эмитента считыватель проверить не может, потому что это `ML-DSA-65`, а стандарт предписывает классические алгоритмы, и репозиторий называет это устройством, а не ограничением, поскольку постквантовое удостоверение,

несущее классическую подпись ради зелёной галочки, было бы классическим удостоверением. Результат проверки также отображается в проверяемое удостоверение W3C, намеренно не идентификационное: словарь его субъекта замкнут и отвергает поля личности по имени, а идентификатор субъекта есть повериферный дескриптор либо отсутствует. Оба суть форматы, а не модели доверия. **РЕАЛИЗОВАНО**

Карточка слоя: независимая проверка

Что это	Четыре пакета в публичных реестрах: самостоятельный верификатор, два SDK и верификатор OpenID4VP; исполнитель соответствия по 118 опубликованным случаям; нормативная спецификация формата передачи; замороженный набор версии 1 с межверсионным набором тестов; фаззер, набор атак и три мутационных учения; два формата-моста.
Зачем оно существует	Проверка, выполняемая эмитентом, просит доверяющую сторону довериться серверу эмитента. Подлинность обязана быть разрешима всяким, где угодно, при выключенной сети и программами, которые автор не писал.
Чему доверяет	Опубликованным ключам эмитентов, которым доверяющая сторона решила доверять (её набору якорей); стандартной библиотеке ML-DSA, а на пути OpenID4VP, зарегистрированному якорю доверия X.509 и ES256.
Чему отказывает	Любому коду или базе данных Polaris; любому входу, который не способен разобрать; любому объекту, поданному верификатору не того типа; любой версии, о которой ему не сказали; запуску без объявленного режима криптографии; одноразовому числу или аудитории, которых он сам не запрашивал.
Что видит	Предмет и набор якорей. Для SD-JWT VC, ровно раскрытые утверждения. Больше ничего.
Чего не видит	Сохраняются ли у удостоверения полномочия, если вместе с ним не предъявлен предмет о состоянии; 43 поля вердикта, которых опубликованные случаи не стесняют.
Если откажет	Ошибочно принятая мутация роняет фаззер; отказ, переставший отказывать, роняет мутационное учение SDK; расхождение спецификации и подписанта роняет <code>check_wire_spec_matches_code</code> ; пакет, который посторонний не может установить, роняет учение о границе; дефект, мимо которого прошёл каждый внутренний инструмент, есть то, ради чего внешнее и нужно.
Кем проверяется	<code>check_detached_verifier</code> , <code>check_conformance_suite</code> , <code>check_typescript_sdk</code> , <code>check_verifier_fuzz</code> , <code>check_attacks_run</code> , <code>check_protocol_versioning</code> , <code>check_sdk_refusals_are_mutation_tested</code> , <code>check_conformance_asks_the_relying_party_question</code> , <code>check_oid4vp_verifier_boundary</code> , <code>check_documented_verifier_commands_run</code> .
Сейчас или потом	Сейчас, для программ и для одного стороннего кошелька на одном пути. Сертификация, второй кошелёк и реализация родного протокола кем-то ещё суть потом и не в нашей власти.
Внутри и снаружи	Внутри: подписи. Снаружи: слой состояния, отвечающий на вопрос, на который подпись ответить не может.

Подписание удостоверения доказывает, кто его подписал. Оно не доказывает, сохраняются ли у этого удостоверения полномочия: печать на отозванном паспорте столь же подлинна, как печать на действительном. Поэтому Polaris нужно состояние.

7 Проверка не есть полномочия: состояние, отзыв и размен автономности

7.1 Два результата, никогда не один

Весь продукт виден в одном объекте. Доверяющая сторона спрашивает эмитента об удостоверении и получает для отозванного токена:

```
POST /api/v1/verify -> {
  "authentic": true,           // подпись ML-DSA подлинная
  "currently_authoritative": false, // но этот токен отозван
  "status": "REVOKED",
  "usable": false,           // подлинный И полномочный -> false
  "decision": "reject"
}
```

Подлинность постоянна и поддаётся кэшированию. Полномочия свежи и отзывны. Доверяющей стороне нужны оба, и API отказывается схлопывать их в один флаг, потому что подпись остаётся подлинной навсегда, тогда как полномочия под ней могут быть отозваны в любой миг. **РЕАЛИЗОВАНО**

Конечная точка для доверяющей стороны опознаёт вызывающего как организацию, а не как человека, через выдачу учётных данных клиента по OAuth2; она принимает доказательство владения, а не последовательный идентификатор, так что перебрать токены через неё нельзя; она возвращает единообразный вердикт при несовпадении, так что она не есть оракул существования; и она не хранит записи о том, кто кого проверял. **РЕАЛИЗОВАНО**

7.2 Свежее состояние, а не устаревшая реплика

Конечная точка проверки направляется на читающую реплику ради пропускной способности, и история репозитория хранит ту ошибку, которая расщепила два результата навсегда. Подлинность есть свойство неизменяемого материала, подписанного значения, байтов подписи и хранимого ключа, так что отстающая реплика не способна ни подделать её, ни отрицать. Пригоден ли токен *сейчас*, есть вопрос о текущих полномочиях, а отзыв приземляется на ведущий узел; реплика внутри своего окна отставания могла бы всё ещё показывать только что отозванный токен действующим. Начиная с v9.264 состояние, решающее **usable**, закреплено за ведущим узлом, ответ называет свой источник и несёт **as_of** и нулевую границу отставания, а вызывающий вправе кэшировать подлинность и обязан перечитывать полномочия. **РЕАЛИЗОВАНО**

7.3 Отзыв

Отзыв есть переход состояния через одну дозволенную процедуру, которая в той же транзакции пишет и в **RevocationList**, так что список, обращённый к верификатору, никогда не расходится с состоянием токена, а триггер отвергает всякий иной путь в **REVOKED**. Список несёт канонический словарь причин; след аудита несёт процедурную подробность и соподписанта, когда потолок усмотрения эмитента его потребовал. Отзыв смотрит вперёд: события, записанные, пока удостоверение было действительным, не обесмысливаются, потому что история не переписывается. **РЕАЛИЗОВАНО**

7.4 Полномочия при выключенной сети

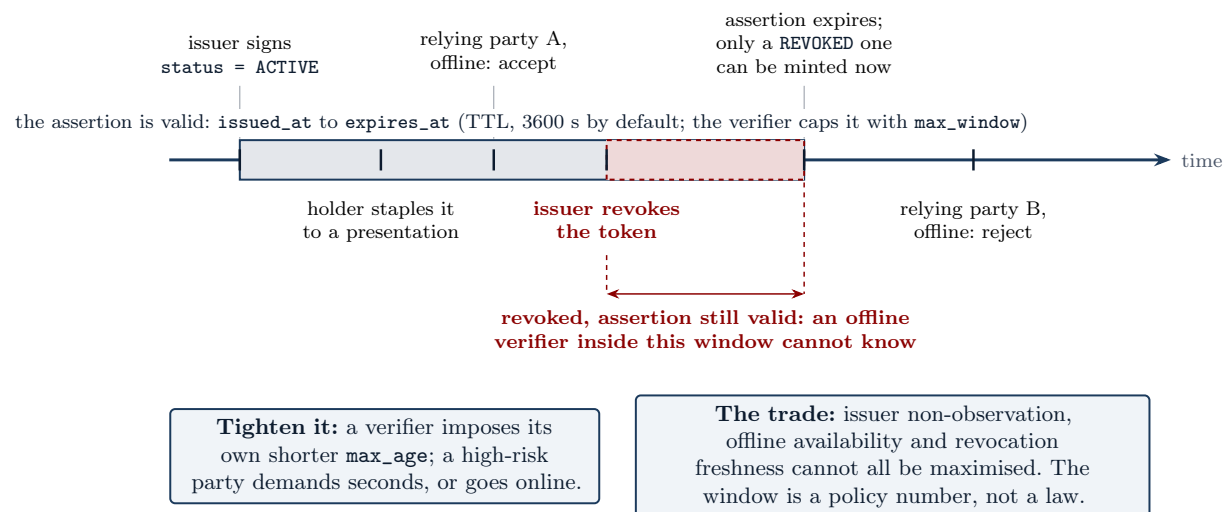


Рис. 7: Размен автономности, нарисованный как временная шкала. Эмитент подписывает утверждение о состоянии с окном; держатель подписывает его к предъявлению; доверяющая сторона, проверяющая автономно внутри окна, принимает. Если эмитент отзовёт удостоверение внутри окна, последнее утверждение **ACTIVE** останется действительным до истечения, и автономный верификатор об этом знать не может. Окно ограничено сроком жизни, выданным эмитентом (по умолчанию час), и величиной `max_window`, которую навязывает верификатор; доверяющая сторона с высоким риском требует секунд либо выходит в сеть.

Автономный верификатор не способен спросить эмитента. Polaris отвечает на это короткоживущим подписанным утверждением о состоянии: подпись эмитента тем же ключом, которым подписано удостоверение, над текущим состоянием токена и окном. Держатель забирает его, предъявляя подлинное удостоверение, без предъявителя и без личных данных, и подписывает к предъявлению. Верификатор принимает автономно, только если удостоверение подлинно, утверждение подлинно, связано именно с этим удостоверением, свежо по собственным часам верификатора, не шире окна, которое верификатор примет, и равно **ACTIVE**. Правило, что верификатор никогда не принимает удостоверения позже чем через одно окно после его отзыва и что окном является то, которое задал верификатор, а не то, которое вычеканил эмитент, есть теперь формальная модель, проверяемая при каждой отправке с конфигурацией, на которой она обязана упасть. **РЕАЛИЗОВАНО**

Цена изложена так же прямо, как и выгода. Поскольку проверка не касается эмитента, эмитент никогда не узнаёт, что она была: автономно приватнее, чем в сети. И поскольку она не касается эмитента, последнее утверждение отозванного удостоверения остаётся действительным до истечения. Ненаблюдаемость эмитентом, автономная доступность и свежесть отзыва не могут быть доведены до предела одновременно; развёртывание выбирает две, а окно есть число политики, а не закон. Одно следствие записано в сводной таблице как известное ограничение, а не исправлено: эмитент чеканит часовое окно, собственная граница по умолчанию у эталонного верификатора не задана, а потолок, который несёт сам предмет, проверкой утверждения не читается, так что предельный автономный срок жизни состояния держится отчасти на настройках доверяющей стороны. Доверяющая сторона, которой нужны секунды, задаёт секунды либо выходит в сеть.

7.5 Состояние между органами и через сети, которым не доверяют

Ещё два подписанных объекта переносят состояние между органами без обратного вызова, который эмитент мог бы записать. Контрольная точка эпохи есть обязательство органа о последней точке его цепи эпох: номер эпохи, её корень Меркла и предыдущая эпоха, которую она продолжает; две контрольные точки доказывают монотонность, а два разных корня, подписанные под одним номером эпохи, суть криптографическое доказательство того, что орган двоесказательствовал о собственной истории. Лента отзывов есть упорядоченное множество листьев отозванных удостоверений, каждый из которых есть SHA3-256 значения токена, выводимый лишь держателем; более новая лента, выбросившая опубликованный отзыв, есть пойманный откат, потому что лежащая под ней таблица работает только на добавление. В масштабе федерации сводчик зеркалит ленты и контрольные точки многих органов, дословно и по-прежнему под собственной подписью каждого участника, в один короткоживущий набор. Сводчику не доверяют в части правильности: полностью распоряжаясь набором, пересчитывая обязательство и переподписывая конверт, он всё равно не способен подделать состояние участника, потому что не способен подписаться как участник. Набор добавляет доступность, а не доверие. **РЕАЛИЗОВАНО**

Подписание предмета о состоянии есть также и то, что позволяет недоверенному кэшу его переносить, а кэш вносит тот единственный отказ, которого подписание не предотвращает, а именно время. Начиная с v9.358 директива кэширования никогда не есть константа: **max-age** всякого оконного предмета есть его собственный остаток жизни, вычисленный из подписанного эмитентом **expires_at**, так что кэш не способен пережить окно, к которому эмитент себя обязал; всякий предмет, называющий одно удостоверение, есть **no-store**, потому что общий кэш, держащий такой, выдал бы удостоверение одного держателя другому; а истёкшее или неразбираемое окно не кэшируется вовсе. Раздача в масштабе сети доставки содержимого тем самым описана и отработана в учениях; ни в одном развёртывании её нет. **РЕАЛИЗОВАНО** для предметов, учений и правил кэширования; **ПРЕДЛОЖЕНО** для развёртывания раздачи.

Темп смены эпох есть то единственное число, которое задаёт три не связанных между собой свойства, и проектная запись теперь говорит, какие именно: свежесть отзыва, поскольку отозванное удостоверение выпадает из *следующей* эпохи; размер толпы, поскольку множество анонимности есть эпоха; и то, как часто обнуляется у доверяющей стороны учёт «один человек однажды». Заккрытие эпохи было когда-то дорогой частью, и дорогой по дурной причине: дерево постоянной глубины, дополненное до национальной глубины 24, материализовывало шестнадцать миллионов листьев при любом населении, так что корень над тысячей участников занимал 10,8 секунды и 2,9 ГБ. Начиная с v9.357 дополнение есть одно повторяющееся значение, хешируемое по разу на уровень, и корень над миллионом участников занимает 2,1 секунды в десятках мегабайт; связывающим ограничением на темп есть нижний предел анонимности на контекст, а не вычисления. **РЕАЛИЗОВАНО**

Карточка слоя: проверка и состояние

Что это	Конечная точка проверки для доверяющей стороны, список отзывов, подписанное утверждение о состоянии, контрольные точки эпох, ленты отзывов, сводный набор состояний и правила кэширования, позволяющие недоверенной сети всё это переносить.
Зачем оно существует	Подлинность не есть полномочия. Верификатору нужно знать не только, что утверждение было сделано, но и что оно по-прежнему в силе, а иногда нужно знать это при выключенной сети.
Чему доверяет	Подписывающему ключу эмитента, ради утверждения; собственным часам, ради свежести; ведущей базе данных, ради сетевого вердикта.
Чему отказывает	Мнению реплики о текущем состоянии; утверждению шире потолка верификатора; слову сводчика о состоянии участника; ленте, не связанной с ключом эмитента; записи кэша, которая пережила бы подписанное эмитентом окно.
Что видит	Состояние одного удостоверения в один момент; множество отзыванных листов.
Чего не видит	Отзыв, случившийся после того, как было подписано имеющееся у него утверждение; действующее население, которого не публикует ни одна лента.
Если откажет	Устаревший вердикт ограничен окном, и граница есть граница верификатора; раздвоенная цепь эпох или откаченная лента суть неотрицаемое свидетельство против органа; лгуний сводчик ловится на подписи участника.
Кем проверяется	<code>check_relying_party_api</code> , <code>check_offline_verification</code> , <code>check_epoch_revocation_propagation</code> , <code>check_federation_status_bundle</code> , <code>check_status_distribution</code> , модель свежести состояния, нагрузочные учения проверки при переключении.
Сейчас или потом	Сейчас, для предметов и правил. Раздача в национальном масштабе потом.
Внутри и снаружи	Внутри: верификаторы. Снаружи: граница приватности, решающая, что верификатору дозволено узнать, пока он проверяет.

Состояние доказывает нынешние полномочия, но доказательство способно открыть куда больше, чем верификатору требовалось, и всякая проверка есть факт о чьём-то дне. Следующий слой ограничивает то, что узнаёт кто бы то ни было, и настоящая версия сообщает, что об этой границе было измерено, а не что задумывалось.